Quantstamp DRAFT

Sapien POQ Protocol

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Oracle
Timeline	2026-02-25 through 2026-03-06
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• Sapien-io/sapien-contracts #505c56a
Auditors	• Valerian Callens Senior Auditing Engineer • Roman R Senior Auditing Engineer • Andrei Stefan Auditing Engineer

Documentation quality	Medium
Test quality	Undetermined
Total Findings	43 Unresolved: 43
High severity findings	7 Unresolved: 7
Medium severity findings	21 Unresolved: 21
Low severity findings	11 Unresolved: 11
Undetermined severity findings	1 Unresolved: 1
Informational findings	3 Unresolved: 3

Summary of Findings

Sapien PoQ is a decentralized quality oracle for AI workflows. It enables stake-weighted, consensus-based verification of AI-generated data and agent behaviors. Originators post tasks with funded reward pools, contributors claim specific data points and submit work, validators independently score that work via commit-reveal, and the protocol settles rewards and slashing based on weighted agreement. Participants may be human, AI agents, or hybrid teams.

The documentation is very detailed, but does not always match what the code is doing. The test suite includes several types of tests (unit tests, integration tests, fuzzing). However, it does not offer an easy way to compute the test coverage metrics.

The audit has identified a range of vulnerabilities that pose significant risks to the integrity and functionality of the system. Key issues include methods for malicious actors to reset deposit timestamps, manipulate reputations, and drain escrow funds through exploits in the project's dispute mechanisms. Moreover, mechanisms meant to govern consensus and project fidelity can be gamed.

Recommendations include implementing strategies to bolster the integrity of the commit-reveal mechanisms to prevent collusion and safeguard against timing manipulations are advised, including setting minimum participant thresholds and using more sophisticated randomness sources for validator assignments. Addressing these vulnerabilities and implementing these recommendations are crucial for the security and reliability of the Sapien protocol, ensuring fair and trustworthy interactions for all participants.

The Sapien team was highly engaged and responsive throughout the audit, promptly addressing our questions and participating in productive discussions.

ID	DESCRIPTION	SEVERITY	STATUS
POQ-1	Originator Can Complete Projects While Contributors Hold Active Reserved Claims	• High	Unresolved
POQ-2	Vault's Deposit-Age Protections Bypassed via Share Transfer and Reset via Dust Deposits	• High	Unresolved

ID	DESCRIPTION	SEVERITY	STATUS
POQ-3	Not Storing Contributions per Nonce Enables Cross-Nonce Reward Theft and Dispute Corruption	High	Unresolved
POQ-4	Cancellation Paths Do Not Unwind Active Pipeline, Permanently Stranding Validator Funds	High	Unresolved
POQ-5	`expireClaim()` Double-Accounting Permanently Blocks Consensus and Traps Validator Stakes	High	Unresolved
POQ-6	Missing Reveal Phase Lower Bound Enables Early Score Disclosure	High	Unresolved
POQ-7	Reported Originator Reclaims Escrow After Project Cancellation, Depriving Contributors of Rewards	High	Unresolved
POQ-8	Reputation System Can Be Systematically Farmed via Self-Contained Cycles	Medium	Unresolved
POQ-9	Profitable Self-Dispute Collusion Cycle Extracts Escrow at Zero Net Cost	Medium	Unresolved
POQ-10	`numberOfValidations` = 1 Collapses All Anti-Collusion Guarantees	Medium	Unresolved
POQ-11	Protocol Pausability Creates Slash and Reputation Exposure for Honest Participants	Medium	Unresolved
POQ-12	Unbounded Loops in `removeProject()` and `claimToValidate()` Create Gas-Based Liveness Denial of Service	Medium	Unresolved
POQ-13	Upheld Dispute Does Not Restore Slot Lifecycle or Pay Challenger Consistently	Medium	Unresolved
POQ-14	Non-Outlier Validators Gain Positive Reputation on Upheld Disputes	Medium	Unresolved
POQ-15	Escrow Settlement Is Order-Dependent and Late Claimants Receive Reduced Payouts or Nothing	Medium	Unresolved
POQ-16	`completeProject()` Missing Originator Report Check Enables Slash Evasion	Medium	Unresolved
POQ-17	`removeProject()` / `expireClaim()` Write-Write Conflict Permanently Reverts Claim Expiry	Medium	Unresolved
POQ-18	Index Recycling Destroys Prior Dispute Window and Enables Stale-Nonce Disputes	Medium	Unresolved
POQ-19	Validator Non-Reveal Indefinitely Stalls Consensus and Blocks Project Completion	Medium	Unresolved
POQ-20	Validation Claim Slot Exhaustion	Medium	Unresolved
POQ-21	Disputes Can Be Opened and Processed on Cancelled Projects	Medium	Unresolved
POQ-22	`minClaimAmount` Is Token-Agnostic and Cannot Encode Absolute Value	Medium	Unresolved
POQ-23	Originator Can Claim Validation Slots for Their Own Project	Medium	Unresolved

ID	DESCRIPTION	SEVERITY	STATUS
POQ-24	New Submission Round Starts Before Prior Nonce Challenge Period Expires	Medium	Unresolved
POQ-25	Commit-Reveal Hash Lacks Validator Binding, Enabling Score Replay	Medium	Unresolved
POQ-26	Deterministic PRGN on L2 Enables Selective Validator Assignment	Medium	Unresolved
POQ-27	Asymmetric Reputation Rollback on Dispute Settlement	Medium	Unresolved
POQ-28	Admin Timing Parameter Changes Retroactively Affect in-Flight Commitments	Medium	Unresolved
POQ-29	Commit-Hash Encoding Drift Between Documentation and Contracts Causes Slash Exposure	Low	Unresolved
POQ-30	Admin Changes to `minValidationStake` Penalize Validators Committed Mid-Claim	Low	Unresolved
POQ-31	Custom Reward Tokens Can Bypass Reward Payments and Trick Protocol Participants	Low	Unresolved
POQ-32	Deadline Naming Confusion Grants Validators an Unintended Extended Reveal Window	Low	Unresolved
POQ-33	Disputes and Originator Reports Can Be Front-Run for Profit	Low	Unresolved
POQ-34	`dailyGain` Tracking Inaccurate for Users Near the Reputation Cap	Low	Unresolved
POQ-35	Overflow Handler in `ConsensusLib` Is Unreachable Under Solidity 0.8+	Low	Unresolved
POQ-36	`ORIGINATOR` Role Key Can Collide with a Registered Skill Name	Low	Unresolved
POQ-37	Missing Input Validations on Project and Administrative Parameters	Low	Unresolved
POQ-38	Small Slash Amounts at High Share Prices Round Down to 0	Low	Unresolved
POQ-39	Reputation Decay Timing Allows Day-Boundary Manipulation	Low	Unresolved
POQ-40	Multiple `fundProject()` Calls with Different Adapters Silently Overwrite Prior Adapter	Informational	Unresolved
POQ-41	Hardcoded `VALIDATION_CLAIM_DEADLINE` Cannot Be Tuned without a Contract Upgrade	Informational	Unresolved
POQ-42	Storage Variables Not Initialised in `initialize()`	Informational	Unresolved
POQ-43	Enum Type Confusion Allows Emission of `ValidationClaimExpired()` Events	Undetermined	Unresolved

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
>
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/Sapien-io/sapien-contracts`

Included Paths: `src`

Operational Considerations

1. Operator liveness is a hard protocol requirement: `escalateDispute()` and `escalateOriginatorReport()` are permissionless after 7 days and unconditionally uphold with no on-chain protection. There is no cooldown between report rejections, so a griefer can force continuous 7-day windows. Operators must maintain 24/7 responsiveness and actively call expiration functions to prevent stale state.
2. Validator assignment randomness is exploitable by MEV: `claimToValidate()` uses `block.prevrandao` on L2, which MEV searchers can simulate to cherry-pick favorable contribution assignments before submitting.
3. Stake-lock amounts drift from share counts as PPS rises: Slashing burns shares without removing vault assets, increasing price-per-share over time and causing `convertToShares(lockedAssets)` to underestimate required share counts in transfer guards and redemption limits.
4. Multi-round funding does not break reward uniformity: `fundProject()` is only callable in `Created` / `Funded` states. Once the first claim transitions the project to `Active`, `totalRewards` and `totalQuantity` are frozen and `rewardRate` is uniform across all contributors.
5. Outbound reward transfers are not balance-checked for fee-on-transfer tokens: `fundProject()` uses a before/after balance pattern for inbound transfers, but `claimReward()` and `refundEscrow()` transfer accounting values directly, which can revert for rebasing or deflationary reward tokens.
6. Reputation penalties are uncapped and asymmetric: Success grants `+10` points (capped at 100/day) while failure deducts `−50` points with no daily cap, meaning one failure erases five successes and a coordinated attacker can destroy a target's reputation in ~190 rejections.
7. Pending contributions have no contributor-initiated timeout: A contribution stuck in `Pending` status cannot be recovered without the operator calling `removeProject()`, which cancels the entire project.
8. Originator can drain escrow before unsettled validators claim rewards: `completeProject()` only checks `pendingContributions == 0`; `refundEscrow()` becomes callable 30 days later regardless of whether validators have settled, leaving slow validators with nothing.
9. Adapter fees are self-capturable: The adapter address is caller-supplied at contribution and validation time, so any participant can designate themselves as the adapter and get the fee intended for front-end partners.

10. Deregistering a skill does not affect existing projects: The skill check is enforced only at `createProject()`; projects using a subsequently deregistered skill continue operating normally.
11. `setClaimCooldown()` and `setMinClaimAmount()` have no upper bounds: An admin can set these to arbitrarily high values, effectively preventing all participants from calling `claimReward()`.
12. Asset decimals must match `ConsensusLib.PRECISION`: A mismatch causes scoring and reward arithmetic to produce incorrect results; this must be validated at deployment and whenever the vault asset changes.
13. Extremes consensus threshold bypass validator input entirely: A threshold near zero unconditionally accepts all contributions. A threshold of 100% unconditionally rejects them, regardless of validator scores.
14. Originator reputation is not verified on-chain: there is no protocol-level barrier preventing a low-reputation originator from creating new projects. Participants must assess originator track record off-chain before committing stake.
15. `rewardToken` is an arbitrary unvetted address: no on-chain whitelist exists. Reward tokens may be fee-on-transfer, rebasing, or malicious contracts. Participants should verify the token contract before interacting with any project.
16. Pausing does not suspend deadline timers: claim, commitment, and dispute windows continue to expire during a pause, potentially causing undeserved slashing or loss of dispute rights upon resumption.
17. `claimToValidate()` enables speculative slot farming: no stake is locked at claim time, so a validator can claim multiple slots and abandon unfavorable assignments at the cost of only a reputation penalty, degrading slot availability for honest participants.
18. Validator reputation is checked at claim time only: a validator whose reputation falls below `minValidatorReputation` after claiming still has their score counted toward consensus with no re-evaluation.
19. `cancelExpiredValidationClaim()` offers no caller incentive: the function is permissionless but provides no reward, so expired claims will rarely be cleaned up in practice, leaving `validationSlots` persistently occupied by stale state.

Key Actors And Their Capabilities

Originators
Create projects via `createProject()`, fund via `fundProject()`, finalize via `completeProject()`, and reclaim remaining escrow via `refundEscrow()` after a 30-day post-completion grace period.

Contributors
Claim contribution slots via `claimToContribute()`, submit work via `contribute()`, and withdraw rewards via `claimReward()`. Subject to stake slashing on expired or rejected submissions. Cannot contribute to self-originated projects.

Validators
Lock vault shares via `lockValidatorCapacity()`, claim randomly-assigned contributions via `claimToValidate()`, commit/reveal validation scores via `commitValidation()` / `revealValidation()`, self-settle via `settleValidator()`. Subject to tiered slashing (10–100%) for outlier scores, or 100% slashing via permissionless `cancelExpiredCommitment()` for non-reveal.

SapienCore: DEFAULT_ADMIN_ROLE
Controls all protocol parameters (fees, deadlines, bonds, stake requirements, decay rate, treasury), manages reputation skills and the `OPERATOR_ROLE`, and can pause/unpause the contract and perform UUPS upgrades with no timelock.

Configurable functions: `setProtocolFee()`, `setOriginationFee()`, `setContributionFee()`, `setValidationFee()`, `setDecayRate()`, `setDisputeBondBps()`, `setOriginatorStakeRequirement()`, `setOriginatorReportBondBps()`, `setMinValidationStake()`, `setTreasury()`, `setMinClaimAmount()`, `setClaimCooldown()`, `setClaimDeadline()`, `setChallengePeriod()`, `setCommitDeadline()`, `setRevealDeadline()`, `setForceSettleDelay()`, `registerSkill()`, `deregisterSkill()`, `pause()`, `unpause()`, `upgradeToAndCall()`.

SapienCore: OPERATOR_ROLE
Sole authority for dispute and originator-report resolution (`resolveDispute()`, `resolveOriginatorReport()`); can cancel projects and slash originators via `removeProject()`; if unavailable for 7 days, escalation functions become permissionlessly callable and auto-uphold.

SapienVault: DEFAULT_ADMIN_ROLE
Configures `minDepositAge`, grants `ENGINE_ROLE` to `SapienCore`, and can pause/unpause the vault and authorize UUPS upgrades with no timelock.

SapienVault: ENGINE_ROLE
Exclusively held by `SapienCore`. Has unrestricted access to lock, unlock, and slash all contributor and validator stake buckets.

Anyone (Permissionless)
Can call `computeConsensus()`, `forceSettleValidator()`, `expireClaim()`, `cancelExpiredValidationClaim()`, `cancelExpiredCommitment()`, `escalateDispute()`, `escalateOriginatorReport()`, `openDispute()`, and `reportOriginator()`.

Privileged Role Attack Vectors
A compromised `OPERATOR_ROLE` can collude to suppress challenge windows, slash legitimate originators, or selectively uphold disputes.
A compromised `DEFAULT_ADMIN_ROLE` can drain all funds via a single and non timelocked UUPS upgrade.

Findings

POQ-1
Originator Can Complete Projects While Contributors Hold Active Reserved Claims

High Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`, `src/libraries/ContributionLib.sol`

Description: `FinalizationLib.completeProject()` accepts `Active` or `Funded` project status, and requires only that `pendingContributions == 0`. However, `pendingContributions` is incremented in `ContributionLib.contribute()`, when work is submitted, not in `ContributionLib.claimToContribute()`, when slots are reserved. As a result, a window exists where contributors have locked stake and reserved slots, yet the originator can immediately call `completeProject()` because `pendingContributions` is still zero.

After completion, `contribute()` reverts with `ProjectNotActive` because the status is now `Completed`. Contributors cannot submit their work. When the claim deadline passes, `expireClaim()` slashes the contributor's stake for failing to submit, even though the originator made submissions impossible. This represents an abuse path where the originator receives free task reservations while contributors bear the slashing risk.

Exploit Scenario:

1. Contributors call `claimToContribute()`, locking their stakes and reserving slots.
2. The originator immediately calls `completeProject()`. `pendingContributions == 0` passes the check; the project is completed.
3. Contributors call `contribute()` but it reverts with `ProjectNotActive`.
4. When claim deadlines pass, calling `expireClaim()` will slash their stakes.

Recommendation: Consider adding a check in `completeProject()` that `proj.availableSlots == proj.totalQuantity` (no slots are outstanding), or verify that no `Claim` structs in `Active` status reference this project.

POQ-2

Vault's Deposit-Age Protections Bypassed via Share Transfer and Reset via Dust Deposits

• High Unresolved

File(s) affected: `src/SapienVault.sol`, `src/libraries/ValidationLib.sol`

Description: The `minDepositAge` mechanism intended to enforce a timelock on vault deposits has two independent bypass vectors.

Sub-issue A - Transfer bypass:

Executing `SapienVault._update()` transfers vault shares but does not set `lastDepositTimestamp` for the receiver. `SapienVault.lockValidatorCapacity()` checks `lastDepositTimestamp[user]`. For an address that received shares via transfer (not a direct deposit), we have `depositTs == 0`, and the check `if (depositTs > 0)` is skipped entirely. A user can deposit, transfer shares to a fresh address, and immediately lock validator capacity, bypassing the intended waiting period.

Sub-issue B - Dust deposit griefing:

Executing `SapienVault._deposit()` unconditionally sets `lastDepositTimestamp[receiver] = block.timestamp` for any deposit amount, regardless of who initiates it. An attacker calls `vault.deposit(1, victim)` for `1 wei` to reset the victim's timestamp, blocking them from `lockValidatorCapacity()` for up to `MAX_MIN_DEPOSIT_AGE` (seven days).

Exploit Scenario:
Exploit Scenario (Sub-issue A):

1. Alice deposits `1000` tokens into the vault. Her `lastDepositTimestamp` is set.
2. Alice transfers all vault shares to Bob via ERC-20 `transfer()`.
3. Bob's `lastDepositTimestamp` remains zero (never set by `_update()`).
4. Bob calls `lockValidatorCapacity()`. The age check sees `depositTs == 0`, skips entirely, and capacity is locked instantly.

Exploit Scenario (Sub-issue B):

1. A target validator deposits tokens and waits for the deposit age to mature.
2. Just before maturity, an attacker calls `vault.deposit(1, targetValidator)`, resetting `lastDepositTimestamp` to `block.timestamp`.
3. The target's deposit age is reset. The attacker repeats this indefinitely.

Recommendation: 1. Set `lastDepositTimestamp[to] = block.timestamp` inside `_update()` when `to != address(0)` (to cover both transfers and mints).

2. Enforce the age check even when `depositTs == 0`. Treat zero as "never deposited", which should fail the check rather than bypass it.
3. For the dust-griefing vector: update `lastDepositTimestamp` only when `msg.sender == receiver`, or require a minimum deposit amount before updating the timestamp.

POQ-3

Not Storing Contributions per Nonce Enables Cross-Nonce Reward Theft and Dispute Corruption

• High Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`, `src/libraries/DisputeLib.sol`, `src/libraries/ValidationLib.sol`, `src/SapienCore.sol`, `src/Types.sol`

Description: The `contributions` mapping is keyed by `(projectId, index)` without a nonce dimension (`Types.sol`), whereas `consensusReports`, `disputes`, and `validatorCommits` are keyed with an additional nonce. When `computeConsensus()` rejects a contribution it increments `submissionNonce` and recycles the index. A new contributor may claim the same index at nonce `N+1`. Once consensus of nonce `N+1` accepts the slot, the `Contribution` struct is overwritten in-place with nonce-`N+1` data.

This creates two distinct exploit paths.

Sub-issue A - Reward theft: A non-outlier validator from the rejected nonce-N round calls `settleValidator(projectId, index, N)`. `FinalizationLib._settleValidatorFor()` reads `consensusReports[projectId][index][N]` (`computed = true`, rejected round) but reads the current `contributions[projectId][index]` which now holds nonce-N+1 data (`status = Accepted`, `challengeEndsAt` from nonce `N+1`). There is no check that the supplied `nonce` equals `contrib.consensusNonce`, so the validator passes all guards and receives rewards from the project escrow. Every non-outlier validator from the rejected round may repeat this independently.

Sub-issue B — Dispute corruption: `SapienCore.resolveDispute()` and `SapienCore.escalateDispute()` read `contrib.consensusNonce` from the mutable `Contribution` struct to look up the dispute record. After slot recycling and nonce-N+1 consensus, `contrib.consensusNonce` stores `N+1`, so the lookup retrieves `disputes[projectId][index][N+1]`, an empty record. The original dispute at nonce `N` is permanently unreachable. The challenger's bond is permanently stranded in `contributorLock` with no recovery path.

Exploit Scenario:

1. Contributor A claims index `0` on project `P` at nonce `0` and submits work.
2. Validators evaluate and reject the contribution. `computeConsensus()` increments `submissionNonce` to one and recycles the slot.
3. Contributor B claims index `0` at nonce `1`, submits work, and consensus accepts it.
4. **Reward theft:** After nonce-one's challenge period elapses, a nonce-zero validator calls `settleValidator(P, 0, 0)`. The function reads `consensusReports[P][0][0]` (computed, rejected round) but reads `contributions[P][0]` (status `Accepted` from nonce one). The non-outlier branch at `FinalizationLib.sol#L101` sees `Accepted` and pays rewards.
5. **Dispute corruption:** If a dispute was opened on nonce zero and nonce-one consensus overwrites `contrib.consensusNonce` to one, `resolveDispute()` looks up `disputes[P][0][1]`, empty. The original dispute at `disputes[P][0][0]` is unreachable.

Recommendation: 1. Add a nonce consistency check in `FinalizationLib._settleValidatorFor()`: require that the supplied `nonce` equals `contrib.consensusNonce`.

2. Pass `nonce` as an explicit parameter in `resolveDispute()` and `escalateDispute()` instead of reading it from the mutable `Contribution` struct.
3. Alternatively, add a nonce dimension to the `contributions` mapping so that each round's state is stored independently.
4. Block index recycling while prior-round disputes or unsettled validators remain outstanding.

POQ-4

Cancellation Paths Do Not Unwind Active Pipeline, Permanently Stranding Validator Funds

High Unresolved

File(s) affected: `src/libraries/OriginationLib.sol`, `src/libraries/DisputeLib.sol`, `src/libraries/FinalizationLib.sol`, `src/libraries/ValidationLib.sol`

Description: When a project is cancelled via `OriginationLib.removeProject()` (`OriginationLib.sol#L131`) or `DisputeLib.upholdOriginatorReport()` (`DisputeLib.sol#L182`), neither function releases validator in-flight stakes. The wind-down loop at `OriginationLib.sol#L150–L162` iterates over contributions and unlocks `Reserved` and `Pending` contributor locks. It does not iterate over `ValidatorCommit` records or call `SapienVault.releaseCommit()` for validators who have already revealed.

After cancellation, all validator recovery paths are blocked:

1. `FinalizationLib._settleValidatorFor()` reverts with `ProjectNotActive` at `FinalizationLib.sol#L59`.
2. `FinalizationLib.forceSettleValidator()` first checks `ConsensusNotReady`, for indices where consensus was computed it falls through to the `ProjectNotActive` revert.
3. `FinalizationLib.cancelExpiredCommitment()` reverts with `AlreadyRevealed` for validators who have already revealed (`FinalizationLib.sol#L220`).

Additionally, `ValidationLib.claimToValidate()` contains no project status check. New validators may claim, commit, and reveal on a cancelled project, only to find all settlement paths blocked in the same way. On top of that, outlier validators escape slashing and reputation penalties through the same mechanism, undermining accountability.

Exploit Scenario:

1. An originator creates and funds a project. Validators claim, commit, and reveal their scores.
2. The project is cancelled via `removeProject()` or `upholdOriginatorReport()`.
3. Each revealed validator calls `settleValidator()`, reverts with `ProjectNotActive`.
4. Each calls `forceSettleValidator()`, blocked by either `ConsensusNotReady` or `ProjectNotActive`.
5. Each calls `cancelExpiredCommitment()`, reverts with `AlreadyRevealed`.
6. The `inFlight` balance for each validator remains permanently locked. `SapienVault.maxWithdraw()` returns total balance minus the locked in-flight amount, leaving that amount unwithdrawable.

Recommendation:

1. Both cancellation paths must iterate over committed validators and call `SapienVault.releaseCommit()` for each in-flight stake before completing cancellation.

2. Add a project status guard to `claimToValidate()`, `commitValidation()`, and `revealValidation()` that reverts when the project is `Cancelled`.

POQ-5

`expireClaim()` Double-Accounting Permanently Blocks Consensus and Traps Validator Stakes

High Unresolved

File(s) affected: `src/libraries/ContributionLib.sol`, `src/libraries/ValidationLib.sol`

Description: When `ContributionLib.expireClaim()` is called on a claim that includes `Pending` contributions (already submitted, awaiting validation), it computes `unlockAmount = stillInFlight * proj.minStakeToClaim` and releases the contributor lock for those slots via `SapienVault.slashAndUnlockContributor()` (`ContributionLib.sol#L229–L231`). However, the contribution's status remains `Pending`, it is not transitioned to a terminal state.

Later, when `ValidationLib.computeConsensus()` runs on those contributions and the result is `Rejected`, the slash at `ValidationLib.sol#L354` calls `SapienVault.slashContributor(contrib.contributor, minStake)`. Because the lock was already depleted by `expireClaim()`, the vault reverts with `InsufficientContributorLock`, permanently blocking `computeConsensus()` for that index. All validator `inFlight` stakes committed to that contribution become trapped.

This issue arises naturally (without adversarial intent) whenever a contributor claims multiple slots, submits some but not all, and the claim deadline expires.

Exploit Scenario:

1. A contributor claims two slots and submits work on one (status `Pending`), leaving one `Reserved`.
2. The claim deadline passes. Anyone calls `expireClaim()`.
3. `expireClaim()` slashes the `Reserved` slot's lock and unlocks the `Pending` slot's lock, but leaves its status as `Pending`.
4. Validators commit and reveal on the still-`Pending` contribution.
5. `computeConsensus()` is called. If the result is `Rejected`, the slash at `ValidationLib.sol#L354` reverts because the contributor lock was already depleted.
6. `computeConsensus()` is permanently blocked. Validator `inFlight` stakes are trapped. `completeProject()` also fails with `ProjectHasActivePipeline`.

Recommendation:

1. `expireClaim()` should transition `Pending` contributions to a terminal state (for example, `Rejected`) and decrement `pendingContributions`, ensuring `computeConsensus()` never runs on an already-unwound contribution.
2. Alternatively, `computeConsensus()` should verify that the contributor lock is sufficient before attempting the slash, and skip the slash if the lock has already been depleted.

POQ-6

Missing Reveal Phase Lower Bound Enables Early Score Disclosure

High Unresolved

File(s) affected: `src/SapienCore.sol`, `src/libraries/ValidationLib.sol`

Description: `ValidationLib.revealValidation()` at `ValidationLib.sol#L203` enforces only an upper bound on reveal timing (`commitTimestamp + commitDeadline + revealDeadline`) with no lower bound requiring the commit phase to complete before reveals are accepted. A validator may commit and reveal in the same block. The `ValidationRevealed` event emitted at `ValidationLib.sol#L217` exposes the `score` field publicly. Later validators who have not yet committed can observe this on-chain score and craft matching commit hashes, undermining the core anti-collusion mechanism of the scheme.

The degraded scheme becomes plaintext voting where later participants copy earlier scores, enabling:

- Herding toward the first revealer's score.
- Manipulation of the weighted average to force acceptance or rejection of contributions.

Exploit Scenario:

1. Validator V1 commits a hash for score 8000 with salt S1.
2. In the same block, V1 calls `revealValidation()`. The `ValidationRevealed` event publicly emits `score = 8000`.
3. Validator V2 observes the event and computes `commitHash = keccak256(abi.encodePacked(uint256(8000), S2))` using the leaked score.
4. V2 commits and reveals the copied score.
5. The consensus weighted average is skewed toward 8,000, potentially shifting the outcome from rejection to acceptance.

Recommendation: Add a reveal-phase gate in `ValidationLib.revealValidation()`:

```
    require(
        block.timestamp >= vc.commitTimestamp + $.commitDeadline,
```

```
        "CommitPhaseActive"
    );
```

This ensures all commits must be submitted before any reveal is accepted.

POQ-7

Reported Originator Reclaims Escrow After Project Cancellation, Depriving Contributors of Rewards

High Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`, `src/libraries/DisputeLib.sol`, `src/libraries/OriginationLib.sol`

Description: After a project is cancelled via `DisputeLib.upholdOriginatorReport()` (`DisputeLib.sol#L182`) or admin `OriginationLib.removeProject()` (`OriginationLib.sol#L131`), the originator's stake is slashed.

However, contributors with `Accepted` work cannot call `FinalizationLib.releaseContributorReward()`, it reverts on `Cancelled` projects at `FinalizationLib.sol#L151`. Still, `FinalizationLib.refundEscrow()` at `FinalizationLib.sol#L264-L288` explicitly permits `Cancelled` status (`FinalizationLib.sol#L272`) and checks only `msg.sender == proj.originator` (`FinalizationLib.sol#L267`). After `PROJECT_COMPLETION_DELAY` (30 days), the punished originator calls `refundEscrow()` and drains all remaining escrow — including rewards earned by contributors whose `Accepted` contributions were never settled.

This creates a perverse incentive: an originator who anticipates being reported profits if `projectEscrow > originatorLockedStake`, since they recover the escrow but lose only the stake.

Exploit Scenario:

1. A project has `Accepted` contributions with unreleased contributor rewards in escrow.
2. A community member reports the originator. The operator upholds the report, originator stake is slashed and project status is set to `Cancelled`.
3. Contributors call `releaseContributorReward()`, reverts with `ProjectNotActive`.
4. After 30 days, the originator calls `refundEscrow(projectId)`. `proj.originator == msg.sender` (never cleared), `proj.status == Cancelled`, and the delay has elapsed, all checks pass. All remaining escrow is drained.
5. Contributors of `Accepted` work receive nothing.

Recommendation: Pre-compute and pre-distribute owed contributor rewards into `pendingRewards` as part of the cancellation flow, before setting the project to `Cancelled`.

POQ-8

Reputation System Can Be Systematically Farmed via Self-Contained Cycles

Medium Unresolved

File(s) affected: `OriginationLib.sol`, `ContributionLib.sol`, `ValidationLib.sol`, `ReputationLib.sol`

Description: The reputation system can be exploited to rapidly accumulate maximum reputation for any registered skill or the `ORIGINATOR_ROLE_KEY` at negligible economic cost, using two attack paths.

Path 1 — Originator reputation (single block):
A single account calls `createProject()`, `fundProject()`, and `completeProject()` in one block. No contribution is required. The originator receives a positive reputation update for `ORIGINATOR_ROLE_KEY`.

Path 2 — Skill reputation (single block via MEV, or multiple blocks):

- EOA A calls `createProject()`, `fundProject()`.
- EOA B calls `claimToContribute()`, `contribute()`.
- EOA A calls `claimToValidate()`, `commitValidation()`, `revealValidation()`, `computeConsensus()`.

Enabling factors (all present in the current codebase):

- Reward tokens may be arbitrary (worthless) ERC-20 contracts, making protocol fees negligible.
- Projects may be completed with zero contributions (enabling Path 1).
- `numberOfValidations` may be set to one, allowing originator and validator to be the same address.
- SAPIEN stake lockups are temporary and returned at the end of each cycle.

Mitigating factors:

- cap for daily gains
- decay mechanism

Protocol participants with inflated reputation gain disproportionate consensus voting weight (consensus weight = `sqrt(stake) * reputation`), enabling them to override honest validators and steer acceptance or rejection of contributions at will.

Recommendation:

1. Whitelist reward tokens to ensure only vetted tokens with real economic value are used.
2. Require at least one accepted contribution before a project can be completed.

3. Set a minimum `numberOfValidations` of three.
4. Prevent the originator from being the validator for their own project.
5. Require a meaningful, non-refundable economic cost per reputation-updating action.

POQ-9

Profitable Self-Dispute Collusion Cycle Extracts Escrow at Zero Net Cost

• Medium Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`, `src/libraries/ConsensusLib.sol`, `src/libraries/DisputeLib.sol`

Description: A colluding group of validators can execute a profitable attack loop against any project with `numberOfValidations = N` (minimum `1`, maximum `10`). The attack requires no operator involvement, `escalateDispute()` is permissionless and auto-upholds after `DISPUTE_RESOLUTION_DEADLINE` (seven days).

Attack steps:

1. `N` Sybil validators claim all validation slots using the zero-cost claim path.
2. They submit identical scores. Because the standard deviation is zero, outlier detection never triggers and no validator is slashed.
3. A Sybil disputer (address `D`) opens a dispute on the intentionally wrong consensus outcome. A bond equal to 10% of `rewardRate` is locked.
4. The attacker waits seven days, then calls `escalateDispute()`, permissionless and available to anyone once the deadline expires. This automatically triggers `upholdDispute()`.
5. D receives the bond back plus 20% of `rewardRate` from escrow.
6. V1–VN call `settleValidator()`, all stake is returned via the non-outlier path.
7. V1–VN receive positive reputation updates despite their consensus being overturned.

As net result: escrow is progressively drained with no economic loss for the attacker, and the validators' reputations are inflated.

Recommendation:

1. Validators whose consensus was overturned by a successful dispute should receive the opposite reputation update.
2. Require operator or multi-sig participation in dispute escalation to prevent fully permissionless auto-uphold.

POQ-10

`numberOfValidations` = 1 Collapses All Anti-Collusion Guarantees

• Medium Unresolved

File(s) affected: `ValidationLib.sol`, `Constants.sol`

Description: With `numberOfValidations = 1`: the standard deviation is always zero (outlier detection cannot trigger), any score at or above `consensusThreshold` causes acceptance regardless of stake weight, and the validator stake weight cancels out of the reward formula (reward is independent of stake amount). The entire validation flow (claim, commit, reveal, `computeConsensus()`) is executable atomically in a single transaction. A single validator can sweep all pending slots in one transaction at negligible cost.

Recommendation: Set a minimum `numberOfValidations` of three at the protocol level. With two validators, the standard deviation can still be manipulated by choosing an appropriately large stake on one side.

POQ-11

Protocol Pausability Creates Slash and Reputation Exposure for Honest Participants

• Medium Unresolved

File(s) affected: `SapienCore.sol`

Description: `DEFAULT_ADMIN_ROLE` can call `SapienCore.pause()`. While paused, all protocol actions are blocked, but internal timers continue running. Specifically:

- A pause of one hour or more: unsubmitted contributor claims become slashable via `expireClaim()`; contributors suffer a reputation penalty.
- A pause of two hours or more: unrevealed validator commitments expire and become fully slashable via `cancelExpiredCommitment()`, validators suffer reputation penalties.

Honest participants lose funds and receive reputation penalties for inaction they were forced into by the pause itself.

Recommendation: Record the total cumulative paused duration and subtract it from all elapsed-time calculations in deadline comparisons, so that paused time does not count against participants.

POQ-12

Unbounded Loops in `removeProject()` and `claimToValidate()` Create Gas-Based Liveness Denial of Service

• Medium Unresolved

File(s) affected: `src/libraries/OriginationLib.sol`, `src/libraries/ValidationLib.sol`

Description: Two core protocol functions contain unbounded loops whose gas cost grows with user-controlled input.

`OriginationLib.removeProject()` iterates over all `totalQuantity` contributions at `OriginationLib.sol#L150` (`for (uint256 i = 0; i < totalQuantity; ++i)`) and over all `pendingIndices` at `OriginationLib.sol#L167-L172`. Because `OriginationLib.fundProject()` allows arbitrary `quantity` additions with no protocol-level cap, a project's `totalQuantity` can grow unboundedly. Each iteration performs storage reads, conditional writes, and external calls to `SapienVault.unlockContributor()`. Testing confirms that `removeProject()` exceeds 30 million gas for sufficiently large projects.

`ValidationLib.claimToValidate()` scans the entire `pendingIndices` array at `ValidationLib.sol#L69` to find eligible contributions. This array grows with project size and is trimmed only during `computeConsensus()`.

Exploit Scenario:

1. An originator calls `fundProject()` with `quantity = 5000`, setting `totalQuantity = 5000`.
2. Admin calls `removeProject()` to cancel the project.
3. The function iterates 5000 times in the contribution cleanup loop plus additional iterations for `pendingIndices`. Gas exceeds the block limit and the transaction reverts.
4. The project cannot be removed. Contributor locks and validator stakes on this project have no admin recovery path.

Recommendation:

1. Implement paginated processing for `removeProject()` that can be completed across multiple transactions.
2. Add a hard cap on `totalQuantity` per project in `fundProject()`.
3. Replace the linear scan in `claimToValidate()` with an indexed queue or cursor-based iteration.

POQ-13
Upheld Dispute Does Not Restore Slot Lifecycle or Pay Challenger Consistently

• Medium Unresolved

File(s) affected: `src/libraries/DisputeLib.sol`, `src/libraries/FinalizationLib.sol`

Description: `DisputeLib.upholdDispute()` has two related accounting and lifecycle inconsistencies.

Sub-issue A - Slot not returned:

When `upholdDispute()` processes an `Accepted` contribution (`DisputeLib.sol#L95-L107`), it decrements `pendingContributions` at `DisputeLib.sol#L105-L107` but does not change `contrib.status` from `Accepted`, does not increment `availableSlots`, does not push the index to `returnStack`, and does not increment `submissionNonce`. The slot is permanently consumed. New `claimToContribute()` calls revert with `NoSlotsAvailable` for that slot.

Sub-issue B - Challenger unpaid on low escrow:

For `Rejected` contributions (`DisputeLib.sol#L108-L125`), `upholdDispute()` pays contributor compensation first (line 114) then attempts the challenger reward (lines 121-124). If `projectEscrow` lies between `compensation` and `compensation + challengerReward`, the contributor receives payment but the challenger does not. This undermines the economic incentive to file disputes.

Exploit Scenario:

Sub-issue A:

1. A contribution is accepted on a project with `totalQuantity = 1`.
2. A dispute is opened and upheld. `pendingContributions` reaches zero but `availableSlots` remains at zero.
3. `claimToContribute()` reverts, no slots are available. The project can never complete.

Sub-issue B:

1. A contribution is rejected. A challenger opens a dispute.
2. Escrow has enough for contributor compensation but not the full payout.
3. `upholdDispute()` pays the contributor but silently skips the challenger reward.
4. The challenger receives their bond back but no reward for a correct challenge.

Recommendation:

1. For upheld disputes on `Accepted` contributions: transition `contrib.status` to a terminal invalidated state, increment `availableSlots`, push the index to `returnStack`, and increment `submissionNonce`.
2. For the payout flow: check the total required amount atomically before any transfer, or implement pro-rata allocation. Emit an event when a payout is skipped.

POQ-14
Non-Outlier Validators Gain Positive Reputation on Upheld Disputes

• Medium Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`, `src/libraries/ReputationLib.sol`, `src/libraries/ValidationLib.sol`

Description: In `FinalizationLib._settleValidatorFor()`, the call to `ReputationLib.update(validator, skill, true, 0)` at `FinalizationLib.sol#L141` is placed outside the dispute-status conditional block at `FinalizationLib.sol#L101–L139`. When a dispute is upheld on an `Accepted` contribution, the validators who approved it were demonstrably incorrect, yet each still receives `SUCCESS_INCREASE` (+10) reputation. Because consensus weight equals `sqrt(stake) * reputation` (see `ConsensusLib.sol#L48`), this creates a compounding feedback loop: incorrect validators grow their influence even when overturned.

- Reputation updates are applied before the challenge period ends. If a dispute later upholds, rolling back these updates is imprecise because intermediate decay operations alter the score between application and rollback.
- In `computeConsensus()`, contributors whose work is accepted receive a positive reputation bonus. If a dispute subsequently upholds the contribution, the contributor's reputation is reduced via `update(contributor, skill, false, 0)`, but the original bonus from `computeConsensus()` is not subtracted from the reduction, resulting in a net positive reputation change for contributors of disputed work.

Exploit Scenario:

1. A contribution is accepted by consensus. A dispute is opened and upheld, the validators were wrong.
2. Non-outlier validators call `settleValidator()`. They receive zero rewards (correctly blocked by the upheld-dispute check at `FinalizationLib.sol#L110`), but `ReputationLib.update(validator, skill, true, 0)` at `FinalizationLib.sol#L141` still executes.
3. Each validator's reputation increases by `+10`. Over multiple upheld disputes, their consensus weight grows despite consistently incorrect assessments.

Recommendation:

1. Move the positive reputation update at `FinalizationLib.sol#L141` inside the accepted-and-not-upheld conditional, so validators only gain reputation when their assessment was not overturned.
2. Apply contributor and validator reputation updates only after the challenge period expires with no upheld dispute, eliminating the need for imprecise rollback.

POQ-15

Escrow Settlement Is Order-Dependent and Late Claimants Receive Reduced Payouts or Nothing

Medium · Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`

Description: `FinalizationLib._settleValidatorFor()` at `FinalizationLib.sol#L124–L126` caps payouts to the current `projectEscrow` balance without reserving funds for future claimants. Similarly, `FinalizationLib.releaseContributorReward()` at `FinalizationLib.sol#L172–L173` caps to available escrow. `releaseContributorReward()` sets `contrib.rewardReleased = true` at `FinalizationLib.sol#L164` before computing the (possibly capped) payout, making each claim one-shot and final, a contributor who receives a partial payout cannot claim the remainder later.

Because settlement is permissionless and order-dependent, early settlers drain the escrow and late but equally valid claimants receive reduced or zero payouts. The originator may also call `refundEscrow()` after the completion delay to withdraw remaining escrow before all settlements are finalized.

Exploit Scenario:

1. A project has five accepted contributions with validators. Escrow is sufficient for all payouts if claimed simultaneously.
2. Three validators settle immediately after the challenge period, each receiving their full calculated reward.
3. A dispute on another contribution drains additional escrow via upheld-dispute payouts.
4. The remaining two validators call `settleValidator()` but `projectEscrow` is now insufficient. They receive partial or zero payouts.
5. The originator calls `refundEscrow()` after the completion delay, withdrawing leftover escrow before all claims are settled.

Recommendation:

1. Snapshot total liabilities per claimant class at consensus time and reserve those amounts.
2. Implement pro-rata or deterministic allocation independent of transaction ordering.
3. Block `refundEscrow()` until all validator settlements and contributor rewards have been claimed, or earmark reserved amounts at consensus time.

POQ-16

`completeProject()` Missing Originator Report Check Enables Slash Evasion

Medium · Unresolved

File(s) affected: `src/libraries/ContributionLib.sol`, `src/libraries/FinalizationLib.sol`

Description: `FinalizationLib.completeProject()` (`FinalizationLib.sol#L242–L262`) releases `originatorLockedStake` without checking whether an `OriginatorReport` is currently `Open`. The guard `originatorReports[projectId].status != Open` exists in `ContributionLib.claimToContribute()` (`ContributionLib.sol#L52–L54`) to prevent new claims during an active report, but is absent from `completeProject()`.

An originator can front-run `resolveOriginatorReport()` by calling `completeProject()` first, which succeeds as long as `pendingContributions == 0`. This releases `originatorLockedStake` to zero. When the operator subsequently upholds the report, `DisputeLib.upholdOriginatorReport()` reads `originatorLockedStake[projectId]` (now zero) and the slash is a no-op. The originator escapes the full penalty despite the upheld report.

Exploit Scenario:

1. All contributions on the project settle, `pendingContributions == 0`.
2. A community member calls `reportOriginator(projectId, evidenceHash)`, opening a report.
3. The originator observes the pending `resolveOriginatorReport()` transaction in the mempool.
4. The originator front-runs with `completeProject(projectId)`, stake released, balance drops to zero.
5. The operator calls `resolveOriginatorReport(projectId, true)` to uphold, `slashContributor(originator, 0)`, a no-op.

Recommendation: Add an originator report status check to `completeProject()`:

```
if ($.originatorReports[projectId].status == OriginatorReportStatus.Open) {
   revert DisputeInProgress();
}
```

This mirrors the existing guard in `claimToContribute()`.

POQ-17

`removeProject()` / `expireClaim()` Write-Write Conflict Permanently Reverts Claim Expiry

• Medium Unresolved

Description: `OriginationLib.removeProject()` at `OriginationLib.sol#L150-L162` wipes contribution data (`status = Empty`, `claimId = 0`, `contributor = address(0)`) for all `Reserved` and `Pending` slots but does not cancel the parent `Claim` struct. When `ContributionLib.expireClaim()` later iterates the claim's indices at `ContributionLib.sol#L194-L214`, each index is checked with `contrib.claimId != claimId`. Because `removeProject()` zeroed `claimId`, all indices are skipped. The loop ends with `unsubmitted = 0`, but the invariant check at `ContributionLib.sol#L222-L223` computes `expectedUnsubmitted = totalCount - submittedCount`, which may be non-zero. The mismatch triggers a revert with `InvalidIndex`, permanently blocking `expireClaim()`.

Note: contributor stakes are released by `removeProject()` itself. The impact is that the `Claim` struct remains `Active` indefinitely with no cleanup path.

Exploit Scenario:

1. A contributor claims three slots (two `Reserved`, one submitted as `Pending`). Stake is locked.
2. Admin calls `removeProject(projectId)` — stakes are released and `claimId` is set to zero on all three indices.
3. The claim deadline expires. Anyone calls `expireClaim(claimId, [indices])`.
4. The loop checks each index: `contrib.claimId == 0 != claimId` — all three are skipped.
5. `unsubmitted = 0` but `expectedUnsubmitted = 3 - 1 = 2` — revert `InvalidIndex`.
6. `expireClaim()` is permanently blocked. The `Claim` remains `Active` indefinitely.

Recommendation: In `removeProject()`'s wind-down loop, set `claim.status = ClaimStatus.Expired` for each affected `Claim` before wiping contribution data.

POQ-18

Index Recycling Destroys Prior Dispute Window and Enables Stale-Nonce Disputes

• Medium Unresolved

File(s) affected: `src/libraries/ContributionLib.sol`, `src/libraries/DisputeLib.sol`

Description: The `contributions` mapping is nonce-agnostic, creating two related but distinct issues at different points of the slot recycling lifecycle.

Sub-issue A — Prior dispute window destroyed:

When `contribute()` is called on a recycled slot, `ContributionLib.sol#L145` sets `challengeEndsAt = 0`. `DisputeLib.openDispute()` at `DisputeLib.sol#L48` treats `challengeEndsAt == 0` as "consensus never ran" and reverts with `ConsensusNotComputed`. The prior round's dispute window is silently closed the moment a new contributor submits — even if the original `challengeEndsAt` deadline has not yet elapsed. The rejected contributor from the prior round loses the ability to dispute, forfeiting potential compensation and reputation recovery.

Sub-issue B — Stale-nonce dispute locks challenger bond:

`claimToContribute()` recycles a slot but does not clear `consensusNonce` or `challengeEndsAt` left by the prior rejected round. Between `claimToContribute()` and the subsequent `contribute()` call, a challenger may call `openDispute()` using the stale `challengeEndsAt` (still non-zero, still within the prior window). The dispute is recorded at the old nonce. When `contribute()` later sets `challengeEndsAt = 0`, and new consensus sets `consensusNonce = 1`, both `resolveDispute()` and `escalateDispute()` read

`consensusNonce = 1` and look up `disputes[index][1]`, empty. The dispute at nonce zero is permanently unresolvable and the challenger's bond is permanently locked.

Recommendation:

1. Snapshot `challengeEndsAt` and `consensusNonce` per-nonce in the `ConsensusReport` struct so that the shared `Contribution` struct needs not carry them.
2. In `claimToContribute()`, clear both stale fields (`challengeEndsAt = 0`, `consensusNonce = 0`) on the recycled slot.
3. Block re-contribution at a recycled index while the prior round's challenge window is still open.

POQ-19
Validator Non-Reveal Indefinitely Stalls Consensus and Blocks Project Completion

• Medium Unresolved

File(s) affected: `src/libraries/ValidationLib.sol`, `src/libraries/ContributionLib.sol`, `src/libraries/FinalizationLib.sol`

Description: `ValidationLib.computeConsensus()` requires `revealCount >= numberOfValidations` at `ValidationLib.sol#L296`. If one assigned validator commits but never reveals, consensus is permanently blocked for that nonce until the commitment expires and a replacement validator provides the missing reveal.

From the griefing angle, the guaranteed delay per grief round is `commitDeadline + revealDeadline` (default two days). The attacker loses their committed stake (100% slash via `cancelExpiredCommitment()`), but may repeat the cycle indefinitely. `FinalizationLib.cancelExpiredCommitment()` decrements `claimCount` but does not increment `revealCount`, so a complete replace-and-reveal cycle is required after each grief.

From the broader liveness failure angle: even without adversarial intent, if validators never reach quorum for any reason (insufficient eligible validators, widespread failure to reveal), `pendingContributions` is never decremented and `completeProject()` permanently fails. Neither `cancelExpiredValidationClaim()` nor `cancelExpiredCommitment()` decrements `pendingContributions`. There is no protocol-level timeout that forces resolution of a stalled contribution.

Recommendation:

1. Require meaningful collateral or reputation at `claimToValidate()` time to raise the cost of deliberate griefing.
2. Add a contribution-level timeout callable after a configurable deadline from `submittedAt`, which transitions the contribution out of `Pending`, decrements `pendingContributions`, returns the slot to the pool, and unlocks or slashes the contributor's stake.
3. Consider allowing consensus computation with a minimum threshold (for example, two-thirds of validators) once the reveal deadline has expired.

POQ-20 Validation Claim Slot Exhaustion

• Medium Unresolved

File(s) affected: `src/libraries/ValidationLib.sol`

Description: `ValidationLib.claimToValidate()` (`ValidationLib.sol#L51–L121`) requires zero economic stake at claim time. Economic commitment only occurs later at `commitValidation()` time (`ValidationLib.sol#L159`). An attacker with N Sybil addresses, where N equals `numberOfValidations`, maximum 10, can exhaust all validation slots for any contribution, blocking consensus entirely. However, validators risk a part of their reputation.

After `VALIDATION_CLAIM_DEADLINE` (one hour), expired claims are cancellable via `cancelExpiredValidationClaim()` (`ValidationLib.sol#L260`), which resets the reservation and decrements `claimCount`. The attacker may immediately re-claim with the same or different addresses, creating an indefinite grief loop at gas cost.

Exploit Scenario:

1. A contribution at index 0 is `Pending` and awaiting validation. The project has `numberOfValidations = 3`.
2. Three Sybil accounts each call `claimToValidate(projectId, 1)`. `claimCount` reaches three, no legitimate validator can claim.
3. None commit. After one hour, `cancelExpiredValidationClaim()` resets the slots.
4. The attacker re-claims immediately. The cycle repeats indefinitely. The contribution never reaches consensus and the project stalls.

Recommendation:

1. Require a small refundable deposit at `claimToValidate()` time, slashed on expiry and returned on successful `commitValidation()`. This makes Sybil griefing economically costly without burdening honest validators.
2. Alternatively, implement a cooldown or address-level throttle for addresses whose validation claims repeatedly expire without commitment.

POQ-21
Disputes Can Be Opened and Processed on Cancelled Projects

• Medium Unresolved

File(s) affected: `src/libraries/DisputeLib.sol`

Description: `DisputeLib.openDispute()` does not check project status. If consensus was computed during the challenge period before cancellation, a dispute may be opened on a cancelled project. If upheld, the dispute pays challenger rewards from `projectEscrow`, reducing

the escrow available to the originator via `refundEscrow()` after the delay. Reputation changes and extended challenge periods may also occur on a project that is no longer operationally active.

Recommendation: Add a project status check at the start of `openDispute()`:

```
if (proj.status == ProjectStatus.Cancelled) revert ProjectNotActive();
```

POQ-22

`minClaimAmount` Is Token-Agnostic and Cannot Encode Absolute Value

Medium | Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`

Description: `minClaimAmount` is a single `uint256` applied uniformly to all `rewardToken` addresses. A value of `1e18` would block claims of less than `1e18` WBTC while accepting almost any amount of a six-decimal token. The threshold is meaningful for exactly one token denomination but not adapted for all others.

Recommendation: Define `minClaimAmount` as a per-token mapping: `mapping(address => uint256) minClaimAmount`, so each reward token can have an independently adapted threshold.

POQ-23

Originator Can Claim Validation Slots for Their Own Project

Medium | Unresolved

File(s) affected: `src/libraries/ValidationLib.sol`

Description: `ValidationLib.claimToValidate()` prevents a validator from being assigned to contributions they personally submitted but does not prevent an originator from validating for a project they created, which can benefit them economically.

Recommendation: Add `require(proj.originator != msg.sender, "OriginatorCannotValidate")` in `claimToValidate()`.

POQ-24

New Submission Round Starts Before Prior Nonce Challenge Period Expires

Medium | Unresolved

File(s) affected: `src/libraries/FinalizationLib.sol`, `src/libraries/ReputationLib.sol`

Description: When `computeConsensus()` rejects a contribution, it immediately increments `submissionNonce` and pushes the index to `returnStack`, making it available for instant recycling. A new contributor may claim and submit on the same slot while an open dispute on the prior round is still within its challenge window. This overlapping pipeline puts more economic stake at risk than necessary and, combined with other issues, creates additional exploit surface.

Recommendation:

1. On rejection, delay index recycling until the prior round's `challengeEndsAt` has elapsed.
2. Alternatively, block new submissions on a recycled index while any prior-nonce dispute is open.

POQ-25

Commit-Reveal Hash Lacks Validator Binding, Enabling Score Replay

Medium | Unresolved

File(s) affected: `src/libraries/ValidationLib.sol`

Description: The commit hash format is `keccak256(abi.encodePacked(score, salt))`. Because `msg.sender` is not included in the preimage, any validator assigned to the same contribution can copy a previously published `(score, salt)` pair from another validator's on-chain reveal transaction and submit it as their own commit, then immediately copy the reveal. If `V1` has a strong reputation, `V2` can free-ride on `V1`'s judgment without any independent evaluation, collecting rewards and reputation without contributing effort. This undermines the individual accountability assumption of the validation system.

Recommendation: Include `msg.sender` in the commit hash preimage:

```
commitHash = keccak256(abi.encodePacked(validator, score, salt));
```

POQ-26

Deterministic PRGN on L2 Enables Selective Validator Assignment

Medium | Unresolved

File(s) affected: `src/libraries/ContributionLib.sol`

Description: `claimToValidate()` seeds its Fisher-Yates shuffle with `block.prevrandao`. On L1, `prevrandao` is Beacon-Chain RANDAO and a block proposer can bias it by at most one bit at significant cost. However, the documented deployment target is Base (L2), where `prevrandao` is constant across all L2 blocks within the same L2 epoch and equal to the corresponding L1 block's value. This makes the seed deterministic for up to approximately six consecutive L2 blocks.

A validator who knows `prevrandao` for the current epoch can time their `claimToValidate()` call to improve the probability of assignment to a target contribution. A malicious validator can also wrap the call in a conditional revert, reverting if not assigned to the desired contribution and retrying, at gas-only cost.

When only one contribution is pending, the shuffle collapses to a no-op (`seed % 1 = 0`), making assignment entirely deterministic regardless of `prevrandao` .

Recommendation:

1. Increase PRNG entropy beyond `prevrandao` alone (for example, mix in a commitment-based seed, the validator's address, and a block hash).
2. Require a minimum eligible pool depth before accepting `claimToValidate()` calls, ensuring the shuffle has meaningful entropy.
3. Monitor and rate-limit reverted `claimToValidate()` calls in a try-catch with event emission to deter griefing.

POQ-27

Asymmetric Reputation Rollback on Dispute Settlement

Medium | Unresolved

Description: Reputation updates are applied before the challenge period ends. If a dispute subsequently upholds, attempting to roll back those updates is imprecise: intermediate decay operations, applied between the initial update and the rollback, have altered the score, so a simple decrement does not restore the original value. The protocol does not attempt any rollback.

Additionally, in `computeConsensus()` , contributors whose work is accepted receive a positive reputation bonus. If a dispute later upholds that contribution, the contributor's reputation is reduced via `update(contributor, skill, false, 0)` , but the original bonus from `computeConsensus()` is not accounted for in the reduction. A contributor with repeatedly upheld disputes can therefore receive a net positive reputation change from work that was ultimately judged as bad.

Recommendation:

1. Apply contributor and validator reputation updates only after the challenge period expires with no upheld dispute, eliminating the need for rollback entirely.
2. If updates must occur earlier, store the exact delta applied at the time of application and use that stored value to compute the precise rollback.

POQ-28

Admin Timing Parameter Changes Retroactively Affect in-Flight Commitments

Medium | Unresolved

Description: `commitDeadline` and `revealDeadline` are read from global storage at check time, not snapshotted per commitment. `SapienCore.setCommitDeadline()` (`SapienCore.sol#L473`) and `SapienCore.setRevealDeadline()` (`SapienCore.sol#L480`) take immediate effect on all in-flight commitments. Reducing `commitDeadline` from the default one day to the minimum one hour retroactively closes the reveal window for validators who committed more than one hour ago. `FinalizationLib.cancelExpiredCommitment()` , which is permissionless, then slashes 100% of those validators' committed stakes.

Exploit Scenario:

1. Validators commit under the default window (`commitDeadline = 1 day` , `revealDeadline = 1 day` , total window = two days).
2. At T+12h, admin calls `setCommitDeadline(3600)` and `setRevealDeadline(3600)` (one hour each).
3. All validators who committed before T+10h are now expired.
4. Anyone calls `cancelExpiredCommitment()` , the validator's full stake is slashed at 100%.

Recommendation:

1. Snapshot `commitDeadline + revealDeadline` per commitment in the `ValidatorCommit` struct at `commitValidation()` time.
2. Alternatively, apply a timelock or epoch-based delay to all deadline-reduction and stake-minimum increases so in-flight operations are not retroactively penalised.

POQ-29

Commit-Hash Encoding Drift Between Documentation and Contracts Causes Slash Exposure

Low | Unresolved

Description: `ValidationLib.revealValidation()` at `ValidationLib.sol#L208` reconstructs the expected commit hash as `keccak256(abi.encodePacked(score, salt))` where `score` is encoded as `uint256` (32 bytes). However, `docs/architecture/lifecycle.md#L107` specifies the encoding as `keccak256(abi.encodePacked(uint16(score), salt))` — encoding `score` as two bytes. The two hashes differ for all non-trivial score values. A validator following the documentation commits a hash that can never match the on-chain reconstructed hash at reveal time.

`ValidationLib.commitValidation()` at `ValidationLib.sol#L138` accepts an arbitrary `bytes32 commitHash` with no encoding enforcement, so the incorrect commit succeeds. After `commitDeadline + revealDeadline`, `FinalizationLib.cancelExpiredCommitment()` at `FinalizationLib.sol#L228` slashes 100% of the validator's committed stake.

Exploit Scenario:

1. A validator client follows `docs/architecture/lifecycle.md#L107` and computes the commit hash using `keccak256(abi.encodePacked(uint16(score), salt))`.
2. The validator calls `commitValidation()` — succeeds (no encoding check).
3. At reveal time, the validator calls `revealValidation(score, salt)`. The contract computes `keccak256(abi.encodePacked(uint256(score), salt))`, which differs from the committed hash.
4. `revealValidation()` reverts.
5. After the deadline, `cancelExpiredCommitment()` slashes the validator's full stake.

Recommendation:

1. Correct `docs/architecture/lifecycle.md#L107` to specify `uint256(score)` instead of `uint16(score)`.
2. Provide an on-chain helper function such as `computeCommitHash(uint256 score, bytes32 salt) returns (bytes32)` for clients to guarantee canonical encoding.
3. Add CI test vectors that verify documentation examples against the contract's encoding.

POQ-30

Admin Changes to `minValidationStake` Penalize Validators Committed Mid-Claim

• Low ⓘ Unresolved

File(s) affected: `src/libraries/ValidationLib.sol`, `src/SapienCore.sol`

Description: `SapienCore.setMinValidationStake()` applies immediately. A validator who called `claimToValidate()` under a lower value may find the new minimum exceeds their locked capacity when they attempt `commitValidation()`. If they cannot commit in time, `cancelExpiredValidationClaim()` slashes the claim and imposes a reputation penalty — despite the original claim being valid under the parameters at claim time.

Recommendation: Snapshot `minValidationStake` per claim at `claimToValidate()` time, or apply a timelock to increases so in-flight claims are not retroactively penalised.

POQ-31

Custom Reward Tokens Can Bypass Reward Payments and Trick Protocol Participants

• Low ⓘ Unresolved

File(s) affected: `OriginationLib.sol`

Description: Function `OriginationLib.createProject()` allows supplying an arbitrary address as the reward token (`config.rewardToken`). This is problematic because:

1. A worthless token could be supplied, without any real value.
2. A custom contract could be supplied that implements ERC-20 functionality, but behaves in unexpected ways (returns contradicting values, calls other contracts, enabling re-entrancy, ...).
3. An ERC-20 token that looks like a legitimate one, tricking contributors and validators.

While no attack path was found that could abuse this type of attacker-controlled data flow, because there is no cross-contamination between reward token related data and other data (strictly confined to `pendingRewards[*][rewardToken]` and `projectEscrow[*][rewardToken]`), we recommend addressing this issue to ensure a secure and fair user experience for all protocol participants.

Recommendation: We recommend employing a whitelist of vetted and common reward tokens, with corresponding management functions.

POQ-32

Deadline Naming Confusion Grants Validators an Unintended Extended Reveal Window

• Low ⓘ Unresolved

File(s) affected: `src/interfaces/ISapienCore.sol`, `src/Constants.sol`, `src/libraries/ValidationLib.sol`, `src/libraries/FinalizationLib.sol`

Description: `commitDeadline` is documented as "duration validators have to commit scores after a contribution is submitted." However, it is also added to the reveal window check in `ValidationLib.revealValidation()` and `FinalizationLib.cancelExpiredCommitment()`:

```
block.timestamp > vc.commitTimestamp + $.commitDeadline + $.revealDeadline
```

This gives a validator `commitDeadline + revealDeadline` time to reveal after their commit, rather than the intended `revealDeadline` alone. The reveal window is effectively doubled relative to the documented intent.

Recommendation:

1. Clarify the intended semantics for each deadline parameter.
2. The reveal window check should reference `vc.commitTimestamp + $.revealDeadline` only, since `commitDeadline` governs when commits must happen after a contribution is submitted, not when reveals must happen after a commit.

POQ-33
Disputes and Originator Reports Can Be Front-Run for Profit

• Low Unresolved

File(s) affected: `src/libraries/DisputeLib.sol`

Description: `DisputeLib.openDispute()` and `reportOriginator()` accept `(evidenceHash, evidenceCid)` without binding them to `msg.sender`, making that transaction exposed to MEV, where someone would copy these values and submit an identical transaction with a higher gas price, claiming the bond return and challenger reward by front-running the legitimate submitter.

Recommendation: Implement a commit-reveal scheme for dispute and report submissions, binding the preimage to `msg.sender` and a domain separator so that the values cannot be replayed by a different caller or for a separate operation.

POQ-34
`dailyGain` Tracking Inaccurate for Users Near the Reputation Cap

• Low Unresolved

File(s) affected: `src/libraries/ReputationLib.sol`

Description: When `currentScore` is near `MAX_REPUTATION`, the actual gain applied to `currentScore` may be less than `gain` (due to the cap check). However, `dailyGain` accumulates the full uncapped `gain`. This causes `dailyGain` to over-represent the actual score change for high-performing users, effectively reducing their remaining allowed daily gain below `MAX_DAILY_GAIN` in subsequent updates during the same day.

Recommendation: Accumulate into `dailyGain` only the amount by which `currentScore` actually increased, not the full requested gain.

POQ-35
Overflow Handler in `ConsensusLib` Is Unreachable Under Solidity 0.8+

• Low Unresolved

File(s) affected: `src/libraries/ConsensusLib.sol`

Description: `ConsensusLib.calculate()` contains a post-multiplication overflow guard:

```
uint256 weightedVariance = wi * diffSquared;
if (diffSquared != 0 && weightedVariance / diffSquared != wi) {
    weightedVariance = type(uint256).max / 4;
}
```

In Solidity 0.8+, unchecked arithmetic overflow causes a `Panic` revert before reaching the conditional. The guard is therefore dead code, the path it protects is unreachable.

Recommendation: Replace the dead check with an explicit overflow-safe computation using `Math.mulDiv()` or place the multiplication in an `unchecked {}` block with a preceding bounds check.

POQ-36
`ORIGINATOR` Role Key Can Collide with a Registered Skill Name

• Low Unresolved

File(s) affected: `src/SapienCore.sol`

Description: The reputation system uses `keccak256("ORIGINATOR")` as `ORIGINATOR_ROLE_KEY`. `SapienCore.registerSkill()` adds names to `registeredSkills` without checking for conflicts with reserved system identifiers. If an admin registers a skill named `"ORIGINATOR"`, the same hash is used for both the system role key and the skill identifier. Projects could then set `requiredSkill = keccak256("ORIGINATOR")`, allowing validators to accumulate "originator reputation" through validation work and breaking the intended separation between role-based and skill-based reputation.

Recommendation: Add a guard in `registerSkill()` preventing registration of any name whose `keccak256` hash equals a reserved system identifier (at minimum, `"ORIGINATOR"`).

POQ-37
Missing Input Validations on Project and Administrative Parameters

Low Unresolved

File(s) affected: `src/libraries/OriginationLib.sol`, `src/libraries/ContributionLib.sol`, `src/libraries/DisputeLib.sol`, `src/SapienCore.sol`

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error:

- In `OriginationLib.createProject()`:
  - `config.minStakeToClaim` can get any value;
  - `config.minValidationStake` can get any value;
  - `config.minValidatorReputation` can get any value;
  - `metadataCid` can get any value and length;
  - `rewardToken` can be any address, not only tokens;
- In `OriginationLib.fundProject()`:
  - `amount` can be very small and very high;
  - `adapter` can be any address, including the originator itself;
- In `ContributionLib.contribute()`:
  - `submissionHash` can be `bytes32(0)`;
  - `dataCid` can get any value and length;
- In `DisputeLib.openDispute()`:
- `evidenceCid` can get any value and length;
- In `SapienCore.registerSkill()`:
  - `name` can get any value and length;
- In `SapienCore.setOriginatorStakeRequirement()`:
  - `amount` can get any value;
- In `SapienCore.setMinValidationStake()`:
  - `amount` can get any value;
- In `SapienCore.setMinClaimAmount()`:
  - `amount` can get any value;
- In `SapienCore.setClaimCooldown()`:
  - `cooldown` can get any value;

Recommendation: Consider adding the relevant checks.

POQ-38
Small Slash Amounts at High Share Prices Round Down to 0

Low Unresolved

File(s) affected: `src/SapienVault.sol`

Description: When the share-to-asset ratio is high (due to accumulated slashings), `convertToShares(assetAmount)` can round down to 0 for small asset amounts. In `_burnShares`, the `if (shares > 0)` guard means no shares are actually burned, but the calling function has already decremented the lock bucket. This creates fakeslashes where the accounting changes but no economic penalty is imposed.

```
// SapienVault.sol:291-296
function _burnShares(address user, uint256 assetAmount) internal {
    uint256 shares = convertToShares(assetAmount);
    if (shares > 0) {
        _burn(user, shares);
    }
    // If shares == 0, nothing is burned but caller already decremented the lock
}
```

Recommendation: Require `shares > 0` and revert if the slash would have no effect, or track lock buckets in share-denominated terms to avoid the rounding mismatch.

POQ-39
Reputation Decay Timing Allows Day-Boundary Manipulation

• Low Unresolved

File(s) affected: `src/libraries/ReputationLib.sol`

Description: Reputation decay uses integer division `elapsed / 1 days`, creating discrete daily steps. At one day minus one second, zero decay is applied; at one day plus one second, a full day of decay is applied. Users can avoid a day's worth of decay by timing transactions to execute just before a day boundary. The binary threshold creates a predictable exploitation opportunity that scales linearly with elapsed time since the last update.

Recommendation: Use a finer time interval (for example, hours) or a continuous fractional decay calculation to eliminate or substantially reduce the timing manipulation surface.

POQ-40
Multiple `fundProject()` Calls with Different Adapters Silently Overwrite Prior Adapter

• Informational Unresolved

File(s) affected: `src/libraries/OriginationLib.sol`

Description: `OriginationLib.fundProject()` may be called multiple times on the same project. Each call with a different adapter address overwrites `originationAdapter[projectId]`. While previously earned adapter fees remain in `pendingRewards[oldAdapter][token]`, the old adapter address is no longer retrievable via `getOriginationAdapter()`. Off-chain systems that track the origination adapter for fee attribution or transparency will observe an incorrect (most-recent-only) value.

Recommendation:

1. Clarify whether multiple fundings with different adapters are intentional.
2. If so, either prevent adapter changes after the first funding, or maintain a history of adapter addresses with a corresponding view function.

POQ-41
Hardcoded `VALIDATION_CLAIM_DEADLINE` Cannot Be Tuned without a Contract Upgrade

• Informational Unresolved

Description: `VALIDATION_CLAIM_DEADLINE = 1 hours` is a compile-time constant in `Constants.sol`, unlike every other protocol deadline (`commitDeadline`, `revealDeadline`, `claimDeadline`, `challengePeriod`), which are configurable storage variables. The one-hour window may be unsuitable for high-latency network conditions or overly generous for L2 deployments, but can be adjusted only via a governance-approved contract upgrade.

Recommendation: Convert `VALIDATION_CLAIM_DEADLINE` to a configurable storage variable with an appropriate setter and bounds (for example, a minimum of 30 minutes and a maximum of 24 hours).

POQ-42 Storage Variables Not Initialised in `initialize()`

• Informational Unresolved

File(s) affected: `src/SapienCore.sol`

Description: `SapienCore.initialize()` does not set default values for `minValidationStake`, `minClaimAmount`, and `claimCooldown`. These remain at zero until an admin explicitly configures them, creating a window between deployment and configuration during which these parameters could be exploited (for example, zero `minValidationStake` allows effectively free validation claims).

Recommendation: Set safe, non-zero default values for all operational parameters in `initialize()`.

POQ-43
Enum Type Confusion Allows Emission of `ValidationClaimExpired()` Events

• Undetermined Unresolved

File(s) affected: `ValidationLib.sol`, `Types.sol`

Description: In solidity/EVM unused memory is always zero initialized and the first value of enums start with zero. Not using special placeholder values of "EMPTY" or "UNUSED" as the first enum entry may lead to type confusion scenarios, where a non-existing or uninitialized object is passed into a function that checks a status being the first enum value (i.e. "Active") could falsely assume to be handle a proper object.

In particular, the following enums violate this principle:

1. `ProjectStatus` : `Created`.
2. `ClaimStatus` : `Active`.

3. `ValidationClaimStatus` : `Active` .

While most functions do revert at some point when being passed such an uninitialized object, `cancelExpiredValidationClaim()` does not. When provided an unused `claimId` it proceeds to set the claim status to `ValidationClaimStatus.Expired` and emit a `ValidationClaimExpired()` event.

The status overwrite in this case does not block other processes as it is just overwritten in `claimToValidate()` . The impact of the event emission is undetermined as it may have off-chain implications that are outside the scope of this audit.

Recommendation: We recommend adding an "Empty" or "Unused" enum field as the first entry into the above listed enums to prevent any such potential type confusions in general and the emission of above mentioned event in particular.

Auditor Suggestions

S1 `ReputationUpdated` Event Emits Post-Decay Score as `oldScore`

Unresolved

File(s) affected: `src/libraries/ReputationLib.sol`

Description: In `ReputationLib.update()` , the `ReputationUpdated` event is emitted with `oldScore` set to the value after decay has been applied, not the value immediately before `update()` was called. Off-chain listeners that expect `oldScore` to represent the pre-update state will observe an incorrect value.

Recommendation: Capture `oldScore` before applying decay, or emit a separate decay event.

S2 `AdapterFeeTooHigh` Error Used for Unrelated Validation Checks

Unresolved

File(s) affected: `src/SapienCore.sol`

Description: `SapienCore.setProtocolFee()` , `setDecayRate()` , and `setOriginatorReportBondBps()` all revert with `AdapterFeeTooHigh` , even though none of these parameters are adapter fees. The misleading error name complicates debugging and monitoring.

Recommendation: Use distinct, accurately named errors for each setter.

S3 Natspec States Slashed Tokens Are Sent to Treasury but Code Burns Them

Unresolved

File(s) affected: `src/interfaces/ISapienVault.sol` , `src/SapienVault.sol`

Description: The natspec for `ISapienVault.slashContributor()` and `ISapienVault.slashValidator()` states: "Slashed tokens are removed from the lock and sent to the treasury." The implementation calls `_burnShares()` , which burns shares. No treasury transfer occurs. Integrators and auditors relying on the interface documentation will have an incorrect understanding of the protocol's economic design.

Recommendation: Clarify the intended behavior and align the natspec with the implementation.

S4 `removeProject()` Natspec Implies Unfunded-only Restriction Not Present in Code

Unresolved

File(s) affected: `src/interfaces/ISapienCore.sol` , `src/libraries/OriginationLib.sol`

Description: The `ISapienCore` natspec for `removeProject()` states: "Remove a project that has not yet been funded". The implementation allows removal of projects in any status except `Cancelled` , including `Funded` , `Active` , and `Completed` .

Recommendation: Update the natspec to accurately reflect the function's actual behavior.

S5 `fundProject()` Natspec States Transition to Active but Code Transitions to Funded

Unresolved

Description: The natspec for `fundProject()` states it "Transitions the project from Created to Active." The code sets the status to `Funded`. The project transitions to `Active` only on the first `claimToContribute()` call.

Recommendation: Update the natspec to reflect the `Funded` intermediate state.

S6 `resolveDispute()` Natspec Incorrectly Describes a New Validation Round

Unresolved

Description: The natspec for `resolveDispute()` states: "If upheld, the consensus outcome is invalidated, and a new validation round begins". The implementation does not initiate a new validation round on uphold. It updates dispute state and emits an event only.

Recommendation: Update the natspec to accurately describe the uphold behavior.

S7 Comment/code Mismatch in `ConsensusLib.calculate()`

Unresolved

File(s) affected: `src/libraries/ConsensusLib.sol`

Description: The inline comment reads "// Apply PRECISION after division to prevent overflow," but the code multiplies by `PRECISION` before dividing:

```
weightedAverage = Math.mulDiv(weightedSum, PRECISION, totalWeight);
```

The comment is misleading about the order of operations.

Recommendation: Update the comment to accurately describe the computation.

S8 `computeConsensus()` Allows Execution on Non-Active Projects

Unresolved

File(s) affected: `src/libraries/ValidationLib.sol`

Description: The project status check in `computeConsensus()` blocks only `Cancelled` status. No direct impact for `Completed` status was identified, but the guard could be tightened to allow only `Active` (or `Active` and `Funded`) status, eliminating edge-case ambiguity.

Recommendation: Restrict `computeConsensus()` to `Active` and optionally `Funded` project status.

S9 `ContributionAdapterFeePaid` Event Emitted in Reward Settlement, Not at Claim Creation

Unresolved

File(s) affected: `src/interfaces/ISapienCore.sol`, `src/libraries/FinalizationLib.sol`

Description: The natspec for `ContributionAdapterFeePaid` states it is "emitted when a contribution adapter fee is paid during claim creation." The event is actually emitted in `FinalizationLib.releaseContributorReward()` at reward settlement, not at claim creation.

Recommendation: Update the event natspec to accurately describe when the event is emitted, or move the emission to the correct function.

S10 `OriginatorReportResolved` Event Not Emitted in `upholdOriginatorReport()`

Unresolved

Description: `OriginatorReportResolved` is emitted in `DisputeLib.rejectOriginatorReport()` but not in `DisputeLib.upholdOriginatorReport()`. Off-chain listeners tracking report resolution events will not observe upheld reports.

Recommendation: Emit `OriginatorReportResolved` in `upholdOriginatorReport()`.

S11 Unlocked Pragma and Best-Practice Violations

Unresolved

Description: All contracts use an unlocked Solidity version pragma (`pragma solidity ^0.8.30`), meaning a future compiler version may silently alter behaviour. Also, the following additional best-practice items were identified:

1. `OriginationLib.sol#L34–L36`: the `config.originator != address(0) && config.originator != msg.sender` conditional may be removed entirely, as `config.originator` is never used after the check.
2. `ContributionLib.contribute()`: the second clause of the status check (`proj.status != ProjectStatus.Funded`) is redundant — a `Claim` object cannot exist for a project that is still in `Funded` state.
3. `ValidationLib.sol#L193`: the magic number `10_000` should be extracted to a named constant.

4. `ValidationLib.claimToValidate()` : the loop at lines L108-L111 can be merged into the loop at L97-L104 for improved readability and a minor gas saving.
5. `ValidationLib.sol#L328` : the magic number `20` should be extracted to a named constant.

Recommendation:

1. Lock the Solidity pragma to a specific version.
2. Address the identified best-practice items.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/Sapien-io/sapien-contracts`

- `cc6...f16 ./src/Constants.sol`
- `372...259 ./src/SapienCore.sol`
- `c95...ea8 ./src/SapienVault.sol`
- `67e...721 ./src/Types.sol`
- `52c...5d0 ./src/interfaces/ISapienCore.sol`
- `045...b13 ./src/interfaces/ISapienVault.sol`
- `da8...7b9 ./src/libraries/ConsensusLib.sol`
- `803...6f6 ./src/libraries/ContributionLib.sol`
- `032...077 ./src/libraries/DisputeLib.sol`
- `0bb...7cf ./src/libraries/FinalizationLib.sol`
- `572...24c ./src/libraries/OriginationLib.sol`
- `7c2...a36 ./src/libraries/ReputationLib.sol`
- `a84...6e5 ./src/libraries/ValidationLib.sol`

Test Suite Results

All 682 tests executed successfully. Tests were run with `forge test`.

```
No files changed, compilation skipped

Ran 7 tests for test/ConsensusLib.t.sol:ConsensusLibTest
```

```
[PASS] test_calculate_minReputationFloor() (gas: 21539)
[PASS] test_calculate_outlierDetection() (gas: 37781)
[PASS] test_calculate_reputationWeighting() (gas: 21723)
[PASS] test_calculate_singleInput() (gas: 17231)
[PASS] test_calculate_stakeWeighting() (gas: 21867)
[PASS] test_calculate_uniformScores() (gas: 30110)
[PASS] test_calculate_weightedAverage() (gas: 25979)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 233.40ms (79.71ms CPU time)

Ran 2 tests for test/findings/2026-02-
23/POC_004_CancelledProjectEscrowStranding.t.sol:POC_004_CancelledProjectEscrowStranding
[PASS] test_originatorReportCancellationStrandsEscrow() (gas: 809616)
[PASS] test_removeProjectCancellationStrandsEscrow() (gas: 658837)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 246.10ms (8.39ms CPU time)

Ran 12 tests for test/coverage/CoverageGaps.t.sol:QEClaimBranchTest
[PASS] test_claimToContribute_noAdapter() (gas: 313397)
[PASS] test_claimToContribute_noMinStake() (gas: 866639)
[PASS] test_expireClaim_allSubmitted() (gas: 743629)
[PASS] test_expireClaim_noStake() (gas: 775688)
[PASS] test_revert_claimToContribute_exceedsMax() (gas: 34668)
[PASS] test_revert_claimToContribute_wrongProjectStatus() (gas: 274901)
[PASS] test_revert_claimToContribute_zeroQuantity() (gas: 34365)
[PASS] test_revert_contribute_deadlinePassed() (gas: 599452)
[PASS] test_revert_contribute_indexNotInClaim() (gas: 572577)
[PASS] test_revert_contribute_indexNotReserved() (gas: 598898)
[PASS] test_revert_expireClaim_indicesLengthMismatch() (gas: 410402)
[PASS] test_revert_expireClaim_wrongClaimId() (gas: 579508)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 248.50ms (9.24ms CPU time)

Ran 2 tests for test/findings/2026-02-
23/POC_005_CommitHashEncodingMismatch.t.sol:POC_005_CommitHashEncodingMismatch
[PASS] test_uint16PackedCommitHashFailsReveal() (gas: 1474900)
[PASS] test_uint256PackedCommitHashSucceedsReveal() (gas: 1586738)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.19ms (648.13µs CPU time)

Ran 5 tests for test/findings/2026-02-
23/fixes/FIX_2026_02_23_ExpectedBehavior.t.sol:FIX_2026_02_23_ExpectedBehavior
[PASS] testFIX_cancelledFundedProjectCanRefundEscrow() (gas: 658313)
[PASS] testFIX_cannotCommitAfterValidationClaimExpiry() (gas: 2337908)
[PASS] testFIX_expiredValidationClaimsReleaseSlots() (gas: 2102268)
[PASS] testFIX_uint256PackedCommitHashRevealsSuccessfully() (gas: 1583665)
[PASS] testFIX_upheldDisputeOnAcceptedContributionDoesNotDeadlockCompletion() (gas: 3332255)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 252.30ms (14.50ms CPU time)

Ran 3 tests for test/lifecycle/Lifecycle.t.sol:LifecycleHappyPathTest
[PASS] test_fullHappyPathLifecycle() (gas: 2896503)
[PASS] test_multiContributionSingleClaim() (gas: 5273255)
[PASS] test_twoContributorsDifferentIndices() (gas: 4379195)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 7.20ms (5.42ms CPU time)

Ran 10 tests for test/coverage/CoverageGaps.t.sol:QEDisputeBranchTest
[PASS] test_escalateDispute_onAcceptedContribution() (gas: 2430558)
[PASS] test_escalateDispute_onRejectedContribution() (gas: 2522512)
[PASS] test_resolveDispute_rejectedRejected() (gas: 2415834)
[PASS] test_resolveDispute_rejectedUpheld() (gas: 2495311)
[PASS] test_revert_cancelExpiredCommitment_alreadyRevealed() (gas: 1078719)
[PASS] test_revert_cancelExpiredCommitment_notCommitted() (gas: 561336)
[PASS] test_revert_escalateDispute_notOpen() (gas: 29390)
[PASS] test_revert_openDispute_consensusNotComputed() (gas: 560719)
[PASS] test_revert_openDispute_noChallengeEndsAt() (gas: 37333)
[PASS] test_revert_resolveDispute_notOpen() (gas: 34126)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 5.59ms (3.93ms CPU time)

Ran 16 tests for test/lifecycle/Lifecycle.t.sol:LifecycleEdgeCaseTest
[PASS] test_ghostValidatorSlash() (gas: 925441)
[PASS] test_revert_cancelCommitmentBeforeDeadline() (gas: 962939)
[PASS] test_revert_claimExceedsAvailableSlots() (gas: 112486)
[PASS] test_revert_claimZeroReward() (gas: 33438)
[PASS] test_revert_consensusInsufficientReveals() (gas: 1512761)
[PASS] test_revert_contributeWrongClaim() (gas: 572386)
[PASS] test_revert_doubleCommit() (gas: 973410)
```

```
[PASS] test_revert_doubleConsensus() (gas: 2235489)
[PASS] test_revert_doubleRewardRelease() (gas: 2796489)
[PASS] test_revert_doubleSettle() (gas: 2427610)
[PASS] test_revert_fundProjectNotOriginator() (gas: 282163)
[PASS] test_revert_invalidRevealHash() (gas: 959733)
[PASS] test_revert_operationsWhenPaused() (gas: 244641)
[PASS] test_revert_originatorContributes() (gas: 148617)
[PASS] test_revert_releaseBeforeChallengePeriod() (gas: 2236997)
[PASS] test_revert_selfValidation() (gas: 597118)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 254.88ms (15.53ms CPU time)

Ran 6 tests for test/lifecycle/Lifecycle.t.sol:LifecycleMultiActorTest
[PASS] test_complexRejectionThenDisputeFlow() (gas: 4756373)
[PASS] test_multipleSequentialClaims() (gas: 6668940)
[PASS] test_partialExpiryThenNewClaim() (gas: 7517181)
[PASS] test_revert_duplicateProject() (gas: 32394)
[PASS] test_stakeWeightedRewardDistribution() (gas: 2757908)
[PASS] test_validatorReputationGate() (gas: 1054587)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 257.37ms (17.46ms CPU time)

Ran 5 tests for test/lifecycle/Lifecycle.t.sol:LifecycleKnownIssuesTest
[PASS] test_issue_cancelledProjectEscrowStranding() (gas: 517066)
[PASS] test_issue_lateCommitAllowedAfterValidationClaimExpiry() (gas: 1827380)
[PASS] test_issue_uint256CommitHashEncodingMismatch() (gas: 1066713)
[PASS] test_issue_upheldDisputeCanDeadlockProjectCompletion() (gas: 2859678)
[PASS] test_issue_validationClaimExpiryLocksValidatorSlots() (gas: 1586460)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.74ms (3.07ms CPU time)

Ran 1 test for test/findings/2026-02-
23/POC_001_ValidationClaimExpirySlotLock.t.sol:POC_001_ValidationClaimExpirySlotLock
[PASS] test_expiredValidationClaimsPermanentlyBlockNewValidators() (gas: 2128923)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.82ms (605.29µs CPU time)

Ran 14 tests for test/coverage/CoverageGaps.t.sol:QEProjectValidationTest
[PASS] test_fundProject_additionalFunding() (gas: 619129)
[PASS] test_fundProject_noAdapter() (gas: 538974)
[PASS] test_revert_createProject_tooHighConsensusThreshold() (gas: 30307)
[PASS] test_revert_createProject_tooHighNumberOfValidations() (gas: 30188)
[PASS] test_revert_createProject_tooHighValidatorRewardBps() (gas: 30106)
[PASS] test_revert_createProject_zeroConsensusThreshold() (gas: 30248)
[PASS] test_revert_createProject_zeroNumberOfValidations() (gas: 30140)
[PASS] test_revert_createProject_zeroRewardToken() (gas: 29926)
[PASS] test_revert_fundProject_wrongStatus() (gas: 2830965)
[PASS] test_revert_fundProject_zeroAmount() (gas: 326915)
[PASS] test_revert_fundProject_zeroQuantity() (gas: 354741)
[PASS] test_revert_initialize_zeroAdmin() (gas: 3992562)
[PASS] test_revert_initialize_zeroTreasury() (gas: 3992416)
[PASS] test_revert_initialize_zeroVault() (gas: 3992549)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 261.35ms (23.52ms CPU time)

Ran 9 tests for test/lifecycle/Lifecycle.t.sol:LifecycleExhaustiveWorkflowTest
[PASS] test_batchCommitRevealMultiIndexLifecycle() (gas: 3386873)
[PASS] test_failedRevealFlowRecoversAfterCancellingExpiredCommitment() (gas: 2689586)
[PASS] test_forceSettlePathAfterDelay() (gas: 2788545)
[PASS] test_fullLifecycleCompletionAndEscrowRefund() (gas: 3200000)
[PASS] test_revert_completeProjectWithPendingContribution() (gas: 558453)
[PASS] test_revert_refundEscrowBeforeCompletionDelay() (gas: 69824)
[PASS] test_revert_revealAfterCommitAndRevealWindowElapsed() (gas: 967055)
[PASS] test_validationClaimExpiresAfterPartialCommit() (gas: 1283740)
[PASS] test_validationClaimExpiresWithoutCommit() (gas: 860800)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 6.88ms (5.22ms CPU time)

Ran 10 tests for test/coverage/CoverageGaps.t.sol:QEExplicitBranchTest
[PASS] test_computeConsensus_rejection_explicitPath() (gas: 2762880)
[PASS] test_engine_pause_blocks_operations() (gas: 891432)
[PASS] test_openDispute_onRejectedContribution() (gas: 2926603)
[PASS] test_reputationDecay_inGetReputationScoreCached() (gas: 5050940)
[PASS] test_resolveDispute_rejectedOnAccepted_explicit() (gas: 2952288)
[PASS] test_resolveDispute_rejectedOnRejected_explicit() (gas: 2943601)
[PASS] test_resolveOriginatorReport_rejected_explicit() (gas: 1275032)
[PASS] test_revert_openDispute_challengeEndsAtZero() (gas: 1099250)
[PASS] test_settleValidator_nonOutlier_explicit() (gas: 2939999)
```

```
[PASS] test_vault_pause_unpause_explicit() (gas: 35686)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 8.32ms (6.78ms CPU time)

Ran 1 test for test/findings/2026-02-
23/POC_002_LateCommitAfterValidationClaimExpiry.t.sol:POC_002_LateCommitAfterValidationClaimExpiry
[PASS] test_validatorCanCommitAfterValidationClaimIsExpired() (gas: 2354235)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.01ms (1.67ms CPU time)

Ran 3 tests for test/lifecycle/Lifecycle.t.sol:LifecycleFullProjectTest
[PASS] test_feeBreakdownAccuracy() (gas: 74089)
[PASS] test_processMultipleIndices() (gas: 4079144)
[PASS] test_tokenConservation() (gas: 2808800)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.19ms (2.64ms CPU time)

Ran 10 tests for test/lifecycle/Lifecycle.t.sol:LifecycleOriginatorReportTest
[PASS] test_competingValidatorsDifferentStakes() (gas: 3354446)
[PASS] test_completeProjectFailureAndRecovery() (gas: 5001394)
[PASS] test_contributionExpiryDuringValidation() (gas: 3311565)
[PASS] test_mixedContributionsPartialCompletion() (gas: 6096700)
[PASS] test_originatorReportEscalation() (gas: 815471)
[PASS] test_originatorReportRejectedSlashesReporter() (gas: 764589)
[PASS] test_originatorReportUpheldCancelsProject() (gas: 876026)
[PASS] test_revert_duplicateOriginatorReport() (gas: 714316)
[PASS] test_revert_originatorReportsSelf() (gas: 570605)
[PASS] test_validatorSettlementTimingEdgeCases() (gas: 2788405)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 10.30ms (8.52ms CPU time)

Ran 6 tests for test/coverage/CoverageGaps.t.sol:QEOriginatorReportBranchTest
[PASS] test_reportOriginator_onFundedProject() (gas: 773044)
[PASS] test_resolveOriginatorReport_upheld_noOriginatorStake() (gas: 1297550)
[PASS] test_revert_escalateOriginatorReport_notOpen() (gas: 23296)
[PASS] test_revert_reportOriginator_nonExistentProject() (gas: 30021)
[PASS] test_revert_reportOriginator_wrongStatus() (gas: 2817579)
[PASS] test_revert_resolveOriginatorReport_notOpen() (gas: 28594)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.80ms (1.42ms CPU time)

Ran 1 test for test/findings/2026-02-23/POC_003_UpheldDisputeDeadlock.t.sol:POC_003_UpheldDisputeDeadlock
[PASS] test_upheldDisputeOnAcceptedContributionBlocksProjectCompletion() (gas: 3330289)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.05ms (2.58ms CPU time)

Ran 8 tests for test/coverage/CoverageGaps.t.sol:ConsensusLibCoverageTest
[PASS] test_allAgree_noOutliers() (gas: 39476)
[PASS] test_revert_emptyInput() (gas: 16802)
[PASS] test_singleInput_noOutlier() (gas: 26590)
[PASS] test_tier1Slash() (gas: 45154)
[PASS] test_tier2Slash() (gas: 51257)
[PASS] test_tier3Slash() (gas: 80435)
[PASS] test_tier4Slash() (gas: 80639)
[PASS] test_zeroWeight_fallbackToOne() (gas: 30788)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 1.35ms (1.10ms CPU time)

Ran 4 tests for test/coverage/CoverageGaps.t.sol:DirectInitCoverageTest
[PASS] test_QE_constructorDisablesInitializers() (gas: 3909042)
[PASS] test_QE_fullInitialize() (gas: 4452041)
[PASS] test_SV_constructorDisablesInitializers() (gas: 2377862)
[PASS] test_SV_fullInitialize() (gas: 2585852)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 897.58µs (545.21µs CPU time)

Ran 1 test for test/lifecycle/Lifecycle.t.sol:LifecycleOutlierTest
[PASS] test_outlierDetectionAndSlashing() (gas: 4046884)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.11ms (4.39ms CPU time)

Ran 15 tests for test/coverage/CoverageGaps.t.sol:QERemainingGapsTest
[PASS] test_computeConsensus_noStakeToUnlock() (gas: 2721974)
[PASS] test_computeConsensus_rejection_noStakeToSlash() (gas: 2718640)
[PASS] test_computeConsensus_withRequiredSkill() (gas: 2785466)
[PASS] test_extremeReputationDecay() (gas: 5087345)
[PASS] test_getReputation_initialized() (gas: 2757560)
[PASS] test_openDispute_zeroBondBps() (gas: 2931469)
[PASS] test_reportOriginator_zeroBondBps() (gas: 1255409)
[PASS] test_reputationDecay_duringValidation() (gas: 5201098)
[PASS] test_reputationNoChange() (gas: 2941294)
```

```
[PASS] test_resolveDispute_rejected_unfreezes() (gas: 2969991)
[PASS] test_resolveDispute_upheld_accepted() (gas: 2948620)
[PASS] test_resolveOriginatorReport_rejected() (gas: 1279152)
[PASS] test_resolveOriginatorReport_upheld_withStake() (gas: 1449483)
[PASS] test_settleValidator_outlier_partialSlash() (gas: 4638892)
[PASS] test_settleValidator_outlier_zeroSlash_equalWeights() (gas: 3824073)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 15.20ms (13.62ms CPU time)

Ran 6 tests for test/lifecycle/Lifecycle.t.sol:LifecycleRejectionTest
[PASS] test_claimExpirationPartialSubmission() (gas: 977991)
[PASS] test_claimExpirationZeroSubmissions() (gas: 577181)
[PASS] test_mixedOutcomesSameProject() (gas: 4345040)
[PASS] test_rejectionSlashesContributorAndReturnsIndex() (gas: 2262085)
[PASS] test_rejectionThenResubmission() (gas: 4714056)
[PASS] test_revert_expireClaimBeforeDeadline() (gas: 339623)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 6.06ms (4.46ms CPU time)

Ran 3 tests for test/lifecycle/QualitySeparation.t.sol:QualitySeparation
[PASS] test_contributorRecoversReputationAfterRejections() (gas: 15671369)
[PASS] test_goodContributorOutperformsBadContributor() (gas: 30682075)
[PASS] test_higherQualityScoreYieldsHigherReputationBonus() (gas: 4556419)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 30.38ms (28.74ms CPU time)

Ran 2 tests for test/findings/2026-02-18/two/RISK_004_SybilConsensus.t.sol:RISK_004_SybilConsensus
[PASS] test_sybilOverridesHonestConsensus() (gas: 59725)
[PASS] test_sybilWeightAmplification() (gas: 71703)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.71ms (452.33µs CPU time)

Ran 19 tests for test/coverage/CoverageGaps.t.sol:QEReputationAdminBranchTest
[PASS] test_getConsensusReport_empty() (gas: 33124)
[PASS] test_getDisputeConfig() (gas: 22576)
[PASS] test_getDispute_empty() (gas: 33320)
[PASS] test_getOriginatorReport_empty() (gas: 29249)
[PASS] test_getProjectEscrow() (gas: 21063)
[PASS] test_getReputation_uninitialized() (gas: 31176)
[PASS] test_getRevealCount() (gas: 21200)
[PASS] test_getSubmissionNonce() (gas: 18292)
[PASS] test_isValidatorOutlier_empty() (gas: 24723)
[PASS] test_isValidatorSettled_empty() (gas: 21696)
[PASS] test_reputationDailyGainCap() (gas: 11646953)
[PASS] test_reputationDecay() (gas: 4106503)
[PASS] test_reputationDecay_inConsensus() (gas: 2248758)
[PASS] test_revert_setContributionFee_tooHigh() (gas: 19844)
[PASS] test_revert_setDecayRate_tooHigh() (gas: 18674)
[PASS] test_revert_setDisputeBondBps_tooHigh() (gas: 19646)
[PASS] test_revert_setOriginatorReportBondBps_tooHigh() (gas: 19404)
[PASS] test_revert_setOriginatorStakeRequirement_tooHigh() (gas: 41637)
[PASS] test_revert_setValidationFee_tooHigh() (gas: 19646)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 8.69ms (6.38ms CPU time)

Ran 10 tests for test/lifecycle/Lifecycle.t.sol:LifecycleReputationAndFeesTest
[PASS] test_contributionAdapterFeeOnRelease() (gas: 2796693)
[PASS] test_contributorReputationDecreasesOnRejection() (gas: 2242433)
[PASS] test_contributorReputationIncreasesOnAcceptance() (gas: 2237171)
[PASS] test_escrowAccountingPerIndex() (gas: 2824171)
[PASS] test_originationAdapterFeeCredited() (gas: 21385)
[PASS] test_originatorReputationIncreases() (gas: 32236)
[PASS] test_protocolFeeSentToTreasury() (gas: 12926)
[PASS] test_rewardRateSnapshot() (gas: 572573)
[PASS] test_validatorReputationIncreasesOnAccurateSettle() (gas: 2426122)
[PASS] test_validatorRewardsBounded() (gas: 2793532)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 6.95ms (5.39ms CPU time)

Ran 2 tests for test/findings/2026-02-18/two/RISK_005_EscrowUnderflow.t.sol:RISK_005_EscrowUnderflow
[PASS] test_disputeHasCheckButSettlementDoesNot() (gas: 3307188)
[PASS] test_settlementDeductsWithoutSufficiencyCheck() (gas: 2906495)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.79ms (3.58ms CPU time)

Ran 2 tests for test/findings/2026-02-18/two/RISK_009_GhostCommitments.t.sol:RISK_009_GhostCommitments
[PASS] test_ghostCanRevealLateWithinWindow() (gas: 2467332)
[PASS] test_ghostValidatorFreeOption() (gas: 2399018)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.54ms (1.32ms CPU time)
```

```
Ran 2 tests for test/findings/2026-02-
18/two/RISK_010_FlashLoanConsensus.t.sol:RISK_010_FlashLoanConsensus
[PASS] test_sameBlockDepositAndValidation() (gas: 1705200)
[PASS] test_sameBlockInfluencesConsensusOutcome() (gas: 2572340)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.35ms (1.16ms CPU time)

Ran 8 tests for test/coverage/CoverageGaps.t.sol:QESettleBranchTest
[PASS] test_releaseContributorReward_noAdapter() (gas: 3136221)
[PASS] test_revert_releaseContributorReward_disputeOpen() (gas: 2404827)
[PASS] test_revert_releaseContributorReward_notAccepted() (gas: 555084)
[PASS] test_revert_settleValidator_consensusNotReady() (gas: 562500)
[PASS] test_revert_settleValidator_notParticipated() (gas: 2247050)
[PASS] test_settleValidator_accurate_zeroCommit() (gas: 2100133)
[PASS] test_settleValidator_outlier_slashExceedsStake() (gas: 3749124)
[PASS] test_settleValidator_outlier_zeroSlash_nonzeroCommit() (gas: 3197130)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 8.25ms (6.48ms CPU time)

Ran 4 tests for test/findings/2026-02-18/two/RISK_013_NoTimelockUpgrade.t.sol:RISK_013_NoTimelockUpgrade
[PASS] test_instantCoreUpgrade() (gas: 3927879)
[PASS] test_instantVaultUpgrade() (gas: 2395798)
[PASS] test_nonAdminCannotUpgrade() (gas: 3922221)
[PASS] test_upgradePreservesStateAcrossInstantUpgrade() (gas: 4511175)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.86ms (2.60ms CPU time)

Ran 17 tests for test/coverage/CoverageGaps.t.sol:QEUncoveredFunctionsTest
[PASS] test_completeProject_active() (gas: 2279104)
[PASS] test_completeProject_funded() (gas: 95241)
[PASS] test_completeProject_withOriginatorStake() (gas: 2777398)
[PASS] test_engine_authorizeUpgrade() (gas: 3927483)
[PASS] test_engine_pauseUnpause() (gas: 39282)
[PASS] test_reduceValidatorCapacity() (gas: 73892)
[PASS] test_refundEscrow() (gas: 97360)
[PASS] test_revert_completeProject_notOriginator() (gas: 32381)
[PASS] test_revert_completeProject_wrongStatus() (gas: 71005)
[PASS] test_revert_engine_upgradeNonAdmin() (gas: 3922625)
[PASS] test_revert_refundEscrow_notCompleted() (gas: 33494)
[PASS] test_revert_refundEscrow_notOriginator() (gas: 71538)
[PASS] test_revert_refundEscrow_tooEarly() (gas: 69472)
[PASS] test_revert_refundEscrow_zeroRemaining() (gas: 482188)
[PASS] test_revert_setMinValidationStake_tooHigh() (gas: 40906)
[PASS] test_setMinValidationStake() (gas: 41370)
[PASS] test_viewFunctions() (gas: 560147)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 4.49ms (2.89ms CPU time)

Ran 2 tests for test/findings/2026-02-18/two/RISK_015_DisputeGriefing.t.sol:RISK_015_DisputeGriefing
[PASS] test_griefingDelaysRewardClaim() (gas: 2895265)
[PASS] test_oneWeiBondForTinyContributions() (gas: 2650694)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.29ms (1.89ms CPU time)

Ran 1 test for test/coverage/CoverageGaps.t.sol:QEValidationAdapterFeeTest
[PASS] test_contributionFee_zeroFeeBps() (gas: 3301408)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.63ms (1.02ms CPU time)

Ran 4 tests for test/audit/ReproduceIssues.t.sol:ReproduceIssuesTest
[PASS] test_RISK003_settlementSucceedsOnRejection() (gas: 3120768)
[PASS] test_RISK005_escrowInsufficientForAllValidators() (gas: 3874749)
[PASS] test_RISK006_validatorLockedOutAfterResubmission() (gas: 4544565)
[PASS] test_RISK007_zeroStakeGetsWeight() (gas: 2101058)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 5.24ms (3.94ms CPU time)

Ran 8 tests for test/coverage/CoverageGaps.t.sol:QEValidationBranchTest
[PASS] test_commitValidation_zeroStake() (gas: 1180815)
[PASS] test_revert_commitValidation_belowMinStake() (gas: 861626)
[PASS] test_revert_commitValidation_maxValidationsReached() (gas: 1992484)
[PASS] test_revert_commitValidation_notSubmitted() (gas: 2280466)
[PASS] test_revert_commitValidation_zeroHash() (gas: 826641)
[PASS] test_revert_revealValidation_alreadyRevealed() (gas: 1074320)
[PASS] test_revert_revealValidation_invalidScore() (gas: 951851)
[PASS] test_revert_revealValidation_notCommitted() (gas: 560494)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 4.20ms (2.68ms CPU time)
```

```
Ran 28 tests for test/coverage/CoverageGaps.t.sol:SapienVaultCoverageTest
[PASS] test_getStakeAccount() (gas: 23867)
[PASS] test_maxRedeem_limitsToUnlocked() (gas: 78875)
[PASS] test_revert_commitStake_insufficientCapacity() (gas: 24343)
[PASS] test_revert_commitStake_zeroAmount() (gas: 21126)
[PASS] test_revert_initializeZeroAddress() (gas: 2456916)
[PASS] test_revert_lockContributor_insufficientBalance() (gas: 45113)
[PASS] test_revert_lockContributor_zeroAmount() (gas: 20840)
[PASS] test_revert_lockValidatorCapacity_insufficientBalance() (gas: 46957)
[PASS] test_revert_lockValidatorCapacity_zeroAmount() (gas: 21390)
[PASS] test_revert_releaseCommit_insufficientInFlight() (gas: 24382)
[PASS] test_revert_releaseCommit_zeroAmount() (gas: 20994)
[PASS] test_revert_slashAndUnlock_insufficientLock() (gas: 65325)
[PASS] test_revert_slashContributor_insufficientLock() (gas: 24180)
[PASS] test_revert_slashContributor_zeroAmount() (gas: 21849)
[PASS] test_revert_slashValidator_insufficientInFlight() (gas: 23460)
[PASS] test_revert_slashValidator_zeroAmount() (gas: 20705)
[PASS] test_revert_unlockContributor_insufficientLock() (gas: 24224)
[PASS] test_revert_unlockContributor_zeroAmount() (gas: 21192)
[PASS] test_revert_unlockValidatorCapacity_insufficientCapacity() (gas: 24359)
[PASS] test_revert_unlockValidatorCapacity_zeroAmount() (gas: 22069)
[PASS] test_revert_vault_upgradeNonAdmin() (gas: 2390912)
[PASS] test_slashAndUnlock_onlySlash() (gas: 59224)
[PASS] test_slashAndUnlock_onlyUnlock() (gas: 50554)
[PASS] test_totalStaked() (gas: 29319)
[PASS] test_transferGuard_allowsUnlockedShares() (gas: 86762)
[PASS] test_transferGuard_blocksLockedShares() (gas: 70793)
[PASS] test_vault_authorizeUpgrade() (gas: 2396153)
[PASS] test_vault_pauseUnpause() (gas: 35680)
Suite result: ok. 28 passed; 0 failed; 0 skipped; finished in 1.71ms (982.87µs CPU time)

Ran 1 test for test/lifecycle/LifecycleSimulation.t.sol:LifecycleSimulation
[PASS] test_lifecycleSimulation_10Projects() (gas: 101240751)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 332.29ms (93.21ms CPU time)

Ran 3 tests for test/findings/2026-02-
18/two/NEW_001_ClaimExpiryStakeUnlock.t.sol:NEW_001_ClaimExpiryStakeUnlock
[PASS] test_consensusRevertsAfterPrematureUnlock() (gas: 2837196)
[PASS] test_expiryUnlocksInPipelineStake() (gas: 1379094)
[PASS] test_validatorStakePermanentlyLocked() (gas: 2843152)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.30ms (2.02ms CPU time)

Ran 3 tests for test/lifecycle/ReputationDynamics.t.sol:ReputationDynamics
[PASS] test_inactivityDecayReducesReputation() (gas: 4809567)
[PASS] test_reputationDivergesForAccurateVsOutlierValidators() (gas: 30762407)
[PASS] test_reputationGateBlocksDegradedValidators() (gas: 10779565)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 43.08ms (41.42ms CPU time)

Ran 34 tests for test/coverage/CoverageGaps.t.sol:QEFullPathCoverageTest
[PASS] test_cancelExpiredCommitment() (gas: 1443923)
[PASS] test_claimReward() (gas: 3307881)
[PASS] test_claimToContribute_fromReturnStack() (gas: 3019470)
[PASS] test_escalateOriginatorReport() (gas: 1298062)
[PASS] test_escalateOriginatorReport_withStake() (gas: 1422339)
[PASS] test_event_cancelExpiredCommitment() (gas: 1447075)
[PASS] test_event_rejectDispute() (gas: 2945074)
[PASS] test_event_rejectOriginatorReport() (gas: 790947)
[PASS] test_event_upholdDispute() (gas: 2943977)
[PASS] test_fullAcceptanceAndRelease() (gas: 3302381)
[PASS] test_getAdapterFees() (gas: 21605)
[PASS] test_getReputationScore_viaNonZeroMinReputation() (gas: 1623876)
[PASS] test_reputationScoreCached_allPaths() (gas: 4663067)
[PASS] test_reputationScoreCached_withDecay() (gas: 4115813)
[PASS] test_revert_cancelExpiredCommitment_notExpired() (gas: 1496382)
[PASS] test_revert_claimReward_noReward() (gas: 32759)
[PASS] test_revert_claimToContribute_originatorReportOpen() (gas: 783377)
[PASS] test_revert_commitValidation_insufficientReputation() (gas: 1164235)
[PASS] test_revert_contribute_claimCompleted() (gas: 1080862)
[PASS] test_revert_escalateDispute_tooEarly() (gas: 2925547)
[PASS] test_revert_escalateOriginatorReport_tooEarly() (gas: 1250477)
[PASS] test_revert_initializeZeroAdmin_coveragePath() (gas: 4018550)
[PASS] test_revert_initializeZeroAdmin_directProxy() (gas: 3988136)
```

```
[PASS] test_revert_initialize_zeroTreasury() (gas: 3992724)
[PASS] test_revert_openDispute_alreadyOpen() (gas: 2927433)
[PASS] test_revert_openDispute_noConsensus_nonexistent() (gas: 604907)
[PASS] test_revert_openDispute_ownContribution() (gas: 2771118)
[PASS] test_revert_openDispute_windowClosed() (gas: 3308214)
[PASS] test_revert_releaseContributorReward_alreadyReleased() (gas: 3307439)
[PASS] test_revert_releaseReward_disputeUpheld() (gas: 2957444)
[PASS] test_revert_reportOriginator_duplicate() (gas: 1252836)
[PASS] test_revert_reportOriginator_ownProject() (gas: 1178270)
[PASS] test_setValidationFee_success() (gas: 21923)
[PASS] test_settleValidator_rewardCappedByEscrow() (gas: 3166102)
Suite result: ok. 34 passed; 0 failed; 0 skipped; finished in 116.71ms (114.99ms CPU time)

Ran 5 tests for test/lifecycle/Lifecycle.t.sol:LifecycleAdminTest
[PASS] test_adminConfigChanges() (gas: 84100)
[PASS] test_feeChangesApplyToNewProjects() (gas: 630391)
[PASS] test_revert_feeLimits() (gas: 48643)
[PASS] test_revert_nonAdminFeeChanges() (gas: 19469)
[PASS] test_revert_zeroTreasury() (gas: 20245)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.62ms (1.02ms CPU time)

Ran 7 tests for test/lifecycle/Lifecycle.t.sol:LifecycleDisputeTest
[PASS] test_disputeEscalationAutoUpholds() (gas: 2446510)
[PASS] test_disputeRejectedOnAcceptedContribution() (gas: 2471363)
[PASS] test_disputeUpheldOnAcceptedContribution() (gas: 2504084)
[PASS] test_disputeUpheldOnRejectedContribution() (gas: 2505915)
[PASS] test_revert_contributorDisputesOwnAcceptance() (gas: 2246014)
[PASS] test_revert_disputeAfterChallengePeriod() (gas: 2792499)
[PASS] test_revert_duplicateDispute() (gas: 2402614)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 14.07ms (9.38ms CPU time)

Ran 3 tests for test/findings/2026-02-
18/one/SEC_L_03_NoEvidenceValidation.t.sol:SEC_L_03_NoEvidenceValidation
[PASS] test_nonZeroEvidenceHashAccepted() (gas: 2888563)
[PASS] test_openDisputeRejectsZeroEvidenceHash() (gas: 2732354)
[PASS] test_reportOriginatorRejectsZeroEvidenceHash() (gas: 1083659)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.43ms (1.97ms CPU time)

Ran 5 tests for test/findings/2026-02-18/one/SEC_M_01_VaultPauseBypass.t.sol:SEC_M_01_VaultPauseBypass
[PASS] test_depositBlockedWhilePaused() (gas: 51978)
[PASS] test_depositWorksAfterUnpause() (gas: 72496)
[PASS] test_mintBlockedWhilePaused() (gas: 49898)
[PASS] test_redeemBlockedWhilePaused() (gas: 47200)
[PASS] test_withdrawBlockedWhilePaused() (gas: 47068)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.56ms (217.33µs CPU time)

Ran 2 tests for test/findings/2026-02-18/one/SEC_M_02_MissingEvents.t.sol:SEC_M_02_MissingEvents
[PASS] test_otherAdminSettersStillEmitEvents() (gas: 28295)
[PASS] test_setTreasuryEmitsEvent() (gas: 34188)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.80ms (585.58µs CPU time)

Ran 1 test for test/findings/2026-02-
18/one/SEC_M_03_CancelledProjectContribution.t.sol:SEC_M_03_CancelledProjectContribution
[PASS] test_contributeRevertsOnCancelledProject() (gas: 1335202)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.49ms (303.08µs CPU time)

Ran 3 tests for test/findings/2026-02-18/one/SEC_M_04_NoRevealDeadline.t.sol:SEC_M_04_NoRevealDeadline
[PASS] test_lateRevealCannotFrontRunCancelExpiredCommitment() (gas: 2370010)
[PASS] test_lateRevealRejected() (gas: 2389253)
[PASS] test_revealWithinWindowSucceeds() (gas: 2457827)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.03ms (1.81ms CPU time)

Ran 2 tests for test/findings/2026-02-18/one/SEC_M_06_WrongHashLength.t.sol:SEC_M_06_WrongHashLength
[PASS] test_correctHashDerivationMatches() (gas: 3674)
[PASS] test_verifyStorageLocationSucceeds() (gas: 14257)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.35ms (38.00µs CPU time)

Ran 8 tests for test/SapienCore.t.sol:SapienCoreAdminTest
[PASS] test_pause_unpause() (gas: 37628)
[PASS] test_setAdapterFees_revertsTooHigh() (gas: 19752)
[PASS] test_setDecayRate() (gas: 23748)
[PASS] test_setOriginationFee() (gas: 31843)
```

```
[PASS] test_setProtocolFee() (gas: 24589)
[PASS] test_setProtocolFee_revertsTooHigh() (gas: 19752)
[PASS] test_setTreasury() (gas: 30248)
[PASS] test_setTreasury_revertsZeroAddress() (gas: 20553)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 1.49ms (276.04µs CPU time)

Ran 8 tests for test/SapienCore.t.sol:SapienCoreClaimTest
[PASS] test_claimToContribute() (gas: 1022250)
[PASS] test_claimToContribute_revertsNoSlots() (gas: 594995)
[PASS] test_claimToContribute_revertsOriginatorCantContribute() (gas: 592377)
[PASS] test_contribute() (gas: 1130906)
[PASS] test_contribute_completeClaim() (gas: 1267187)
[PASS] test_contribute_revertsNotOwner() (gas: 869371)
[PASS] test_expireClaim() (gas: 1247594)
[PASS] test_expireClaim_revertsNotExpired() (gas: 870104)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.82ms (1.60ms CPU time)

Ran 6 tests for test/SapienCore.t.sol:SapienCoreConsensusTest
[PASS] test_computeConsensus_accepted() (gas: 2750835)
[PASS] test_computeConsensus_rejected() (gas: 2761331)
[PASS] test_computeConsensus_revertsAlreadyComputed() (gas: 2748719)
[PASS] test_computeConsensus_revertsInsufficientReveals() (gas: 2034351)
[PASS] test_settleValidator_accurate() (gas: 2935483)
[PASS] test_settleValidator_revertsAlreadySettled() (gas: 2932582)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 10.05ms (8.68ms CPU time)

Ran 6 tests for test/SapienCore.t.sol:SapienCoreProjectTest
[PASS] test_createProject() (gas: 335659)
[PASS] test_createProject_revertsDuplicate() (gas: 321201)
[PASS] test_fundProject() (gas: 596808)
[PASS] test_fundProject_adapterFee() (gas: 583173)
[PASS] test_fundProject_protocolFee() (gas: 584006)
[PASS] test_fundProject_revertsNotOriginator() (gas: 326764)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 4.93ms (1.52ms CPU time)

Ran 2 tests for test/SapienCore.t.sol:SapienCoreReputationTest
[PASS] test_defaultReputation() (gas: 30924)
[PASS] test_reputationIncreasesOnProjectCreate() (gas: 319635)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.04ms (235.50µs CPU time)

Ran 5 tests for test/SapienCore.t.sol:SapienCoreRewardTest
[PASS] test_adapterClaimReward() (gas: 605285)
[PASS] test_claimReward() (gas: 3290058)
[PASS] test_claimReward_revertsNoReward() (gas: 32613)
[PASS] test_releaseContributorReward() (gas: 2765948)
[PASS] test_releaseContributorReward_revertsChallengeNotElapsed() (gas: 2750636)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 10.57ms (6.99ms CPU time)

Ran 4 tests for test/SapienCore.t.sol:SapienCoreValidationTest
[PASS] test_commitAndReveal() (gas: 1586744)
[PASS] test_commitValidation_revertsAlreadyCommitted() (gas: 1489973)
[PASS] test_commitValidation_revertsOwnContribution() (gas: 1114964)
[PASS] test_revealValidation_revertsInvalidHash() (gas: 1476966)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 7.12ms (3.31ms CPU time)

Ran 10 tests for test/lifecycle/SkillReputation.t.sol:SkillReputation
[PASS] test_builtSkillRepShiftsConsensusInNextRound() (gas: 23936194)
[PASS] test_contributorRejectionPenaltyIsSkillSpecific() (gas: 2794540)
[PASS] test_crossSkillRepDoesNotInfluenceConsensus() (gas: 13463036)
[PASS] test_crossSkillReputationIsolation() (gas: 2980683)
[PASS] test_defaultRepAllowsNewSkillParticipation() (gas: 1365066)
[PASS] test_highSkillRepDominatesConsensus() (gas: 27217600)
[PASS] test_multiSkillDivergentReputation() (gas: 17085518)
[PASS] test_reputationAccruesUnderProjectSkill() (gas: 2953884)
[PASS] test_skillRepGateBlocksDegradedValidator() (gas: 11632883)
[PASS] test_skillReputationInfluencesConsensusWeight() (gas: 11000970)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 75.10ms (69.83ms CPU time)

Ran 6 tests for test/lifecycle/ThresholdEdgeCases.t.sol:ThresholdEdgeCases
[PASS] test_exactThresholdScoreAccepts() (gas: 2216551)
[PASS] test_justAboveThresholdAccepts() (gas: 2211186)
[PASS] test_justBelowThresholdRejects() (gas: 2220867)
```

```
[PASS] test_mixedScoresAveragingToThreshold() (gas: 2252243)
[PASS] test_stakeWeightShiftsConsensusOutcome() (gas: 2374137)
[PASS] test_unanimousMaxScoreAccepts() (gas: 2221870)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 13.78ms (9.23ms CPU time)

Ran 34 tests for test/upgrade/Upgrade.t.sol:UpgradeTest
[PASS] test_core_adminCanUpgrade() (gas: 3988094)
[PASS] test_core_implementationChangesAfterUpgrade() (gas: 3983844)
[PASS] test_core_multipleSequentialUpgrades() (gas: 8470790)
[PASS] test_core_nonAdminCannotUpgrade() (gas: 3981576)
[PASS] test_core_reinitializeBlockedAfterUpgrade() (gas: 3996371)
[PASS] test_core_reinitializerBlockedOnDirectImpl() (gas: 3966063)
[PASS] test_core_reinitializerIdempotentAfterRun() (gas: 3997023)
[PASS] test_core_reinitializerRunsAfterUpgrade() (gas: 3994180)
[PASS] test_core_upgradeEmitsUpgradedEvent() (gas: 3988617)
[PASS] test_core_upgradeExposesNewFunction() (gas: 3989346)
[PASS] test_core_upgradePreservesClaimData() (gas: 5054890)
[PASS] test_core_upgradePreservesProjectState() (gas: 4577889)
[PASS] test_core_upgradePreservesProtocolConfig() (gas: 4033129)
[PASS] test_core_upgradePreservesRoles() (gas: 3997321)
[PASS] test_core_upgradePreservesVaultReference() (gas: 3995621)
[PASS] test_core_upgradeWithMigrationCalldata() (gas: 3996218)
[PASS] test_vault_adminCanUpgrade() (gas: 2448125)
[PASS] test_vault_depositsWorkAfterUpgrade() (gas: 2542154)
[PASS] test_vault_implementationChangesAfterUpgrade() (gas: 2445277)
[PASS] test_vault_multipleSequentialUpgrades() (gas: 4838271)
[PASS] test_vault_nonAdminCannotUpgrade() (gas: 2441603)
[PASS] test_vault_reinitializeBlockedAfterUpgrade() (gas: 2453732)
[PASS] test_vault_reinitializerBlockedOnDirectImpl() (gas: 2427335)
[PASS] test_vault_reinitializerIdempotentAfterRun() (gas: 2457372)
[PASS] test_vault_reinitializerRunsAfterUpgrade() (gas: 2454586)
[PASS] test_vault_upgradeEmitsUpgradedEvent() (gas: 2448205)
[PASS] test_vault_upgradeExposesNewFunction() (gas: 2450852)
[PASS] test_vault_upgradePreservesAssetAddress() (gas: 2452743)
[PASS] test_vault_upgradePreservesAvailableBalance() (gas: 2476426)
[PASS] test_vault_upgradePreservesERC4626State() (gas: 2464391)
[PASS] test_vault_upgradePreservesRoles() (gas: 2465928)
[PASS] test_vault_upgradePreservesShareBalances() (gas: 2456628)
[PASS] test_vault_upgradePreservesStakeAccount() (gas: 3518646)
[PASS] test_vault_upgradeWithMigrationCalldata() (gas: 2457452)
Suite result: ok. 34 passed; 0 failed; 0 skipped; finished in 13.92ms (9.98ms CPU time)

Ran 25 tests for test/libraries/FinalizationLibFuzz.t.sol:FinalizationLibFuzz
[PASS] testFuzz_cancelExpiredCommitment_afterRevealDeadline() (gas: 1451061)
[PASS] testFuzz_cancelExpiredCommitment_revertsBeforeDeadline() (gas: 1502866)
[PASS] testFuzz_claimReward_respectsCooldown(uint256) (runs: 256, μ: 2571654, ~: 2571664)
[PASS] testFuzz_claimReward_revertsBelowMinimum(uint256) (runs: 256, μ: 137006, ~: 137876)
[PASS] testFuzz_claimReward_revertsNoPending() (gas: 32386)
[PASS] testFuzz_claimReward_validClaim() (gas: 104067)
[PASS] testFuzz_completeProject_revertsNotOriginator(address) (runs: 256, μ: 100703, ~: 100703)
[PASS] testFuzz_completeProject_revertsPendingContributions() (gas: 35236)
[PASS] testFuzz_completeProject_validCompletion() (gas: 153410)
[PASS] testFuzz_forceSettleValidator_afterDelay() (gas: 289650)
[PASS] testFuzz_forceSettleValidator_revertsTooEarly() (gas: 43915)
[PASS] testFuzz_multipleValidatorsSettle_correctRewardDistribution() (gas: 726863)
[PASS] testFuzz_refundEscrow_afterCompletionDelay() (gas: 150545)
[PASS] testFuzz_refundEscrow_revertsBeforeDelay() (gas: 132673)
[PASS] testFuzz_refundEscrow_revertsNotCompleted() (gas: 31865)
[PASS] testFuzz_refundEscrow_revertsNotOriginator(address) (runs: 256, μ: 133932, ~: 133932)
[PASS] testFuzz_releaseContributorReward_afterChallengePeriod() (gas: 98027)
[PASS] testFuzz_releaseContributorReward_revertsBeforeChallengePeriod() (gas: 34881)
[PASS] testFuzz_releaseContributorReward_revertsDoubleRelease() (gas: 96142)
[PASS] testFuzz_releaseContributorReward_revertsNotAccepted(uint256) (runs: 256, μ: 39926, ~: 39954)
[PASS] testFuzz_settleValidator_afterChallengePeriod() (gas: 290101)
[PASS] testFuzz_settleValidator_revertsBeforeChallengePeriod() (gas: 97197)
[PASS] testFuzz_settleValidator_revertsDoubleSettle() (gas: 290090)
[PASS] testFuzz_settleValidator_revertsNoConsensus(uint256) (runs: 256, μ: 41609, ~: 41636)
[PASS] testFuzz_settleValidator_revertsNotCommitted(address) (runs: 256, μ: 49049, ~: 49049)
Suite result: ok. 25 passed; 0 failed; 0 skipped; finished in 574.60ms (743.76ms CPU time)

Ran 4 tests for test/findings/2026-02-
18/one/SEC_H_01_PrematureCompletion.t.sol:SEC_H_01_PrematureCompletion
```

```
[PASS] test_canCompleteAfterAllContributionsFinalized() (gas: 2780599)
[PASS] test_cannotCompleteWithInFlightContributions() (gas: 1082513)
[PASS] test_escrowSafeAfterProperCompletion() (gas: 3284039)
[PASS] test_rejectedContributionsDecrementPipeline() (gas: 2767913)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.23ms (2.87ms CPU time)

Ran 4 tests for test/findings/2026-02-18/one/SEC_H_02_NoForceSettle.t.sol:SEC_H_02_NoForceSettle
[PASS] test_forceSettleAfterDelay() (gas: 2908719)
[PASS] test_forceSettleTooEarlyReverts() (gas: 2733463)
[PASS] test_outlierForceSettled() (gas: 2894738)
[PASS] test_selfSettleStillWorks() (gas: 2903676)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 10.93ms (7.46ms CPU time)

Ran 3 tests for test/findings/2026-02-18/one/SEC_H_03_DisputeGriefLoop.t.sol:SEC_H_03_DisputeGriefLoop
[PASS] test_cannotReopenDisputeAfterRejection() (gas: 2927131)
[PASS] test_cannotReopenDisputeAfterUphold() (gas: 2919682)
[PASS] test_rewardBlockedDuringGriefLoop() (gas: 2941711)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 11.06ms (6.72ms CPU time)

Ran 2 tests for test/findings/2026-02-
18/one/SEC_H_04_ExcessiveDisputePayout.t.sol:SEC_H_04_ExcessiveDisputePayout
[PASS] test_multipleOverturnsDoNotDrainEscrow() (gas: 7289838)
[PASS] test_overturnedRejectionCappedToRewardRate() (gas: 2938960)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 11.69ms (7.68ms CPU time)

Ran 3 tests for test/findings/2026-02-18/one/SEC_L_01_OriginatorIgnored.t.sol:SEC_L_01_OriginatorIgnored
[PASS] test_matchingOriginatorAllowed() (gas: 333223)
[PASS] test_mismatchedOriginatorReverts() (gas: 31263)
[PASS] test_zeroOriginatorStillAllowed() (gas: 332638)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.89ms (584.59µs CPU time)

Ran 1 test for test/findings/2026-02-18/one/SEC_L_02_RedundantZeroCheck.t.sol:SEC_L_02_RedundantZeroCheck
[PASS] test_zeroStakeRevertsBeforeRedundantGuard() (gas: 1347500)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.12ms (700.88µs CPU time)

Ran 16 tests for test/SapienVault.t.sol:SapienVaultTest
[PASS] test_adminCanPause() (gas: 35525)
[PASS] test_cannotWithdrawLockedFunds() (gas: 174795)
[PASS] test_commitStake() (gas: 194756)
[PASS] test_decimalsOffset() (gas: 16324)
[PASS] test_deposit() (gas: 134425)
[PASS] test_lockContributor() (gas: 171654)
[PASS] test_lockContributor_revertsInsufficientBalance() (gas: 143062)
[PASS] test_lockValidatorCapacity() (gas: 169406)
[PASS] test_nonAdminCannotPause() (gas: 19205)
[PASS] test_onlyEngineCanLock() (gas: 132644)
[PASS] test_releaseCommit() (gas: 179116)
[PASS] test_slashContributor() (gas: 180672)
[PASS] test_slashValidator() (gas: 207000)
[PASS] test_unlockContributor() (gas: 171592)
[PASS] test_unlockValidatorCapacity() (gas: 174906)
[PASS] test_withdraw() (gas: 120844)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 2.93ms (1.12ms CPU time)

Ran 20 tests for test/libraries/ContributionLibFuzz.t.sol:ContributionLibFuzz
[PASS] testFuzz_batchContribute_revertsEmptyBatch() (gas: 312816)
[PASS] testFuzz_batchContribute_revertsMismatchedArrays() (gas: 455086)
[PASS] testFuzz_batchContribute_validBatch(uint8) (runs: 256, µ: 1787681, ~: 1482822)
[PASS] testFuzz_claimToContribute_revertsExceedsMaxQuantity(uint256) (runs: 256, µ: 33774, ~: 33624)
[PASS] testFuzz_claimToContribute_revertsNoSlots(uint8) (runs: 256, µ: 652511, ~: 652687)
[PASS] testFuzz_claimToContribute_revertsNonExistentProject(bytes32) (runs: 256, µ: 33051, ~: 33051)
[PASS] testFuzz_claimToContribute_revertsOriginatorContributing() (gas: 34960)
[PASS] testFuzz_claimToContribute_revertsZeroQuantity() (gas: 30281)
[PASS] testFuzz_claimToContribute_validQuantity(uint8) (runs: 256, µ: 835677, ~: 738376)
[PASS] testFuzz_contribute_revertsAfterDeadline(uint256) (runs: 256, µ: 316590, ~: 316610)
[PASS] testFuzz_contribute_revertsDoubleSubmission() (gas: 574122)
[PASS] testFuzz_contribute_revertsIndexNotInClaim(uint256) (runs: 256, µ: 638677, ~: 638677)
[PASS] testFuzz_contribute_revertsNotClaimOwner(address) (runs: 256, µ: 312900, ~: 312900)
[PASS] testFuzz_contribute_validSubmission(bytes32,string) (runs: 256, µ: 525756, ~: 525626)
[PASS] testFuzz_expireClaim_revertsAlreadyCompleted() (gas: 528341)
[PASS] testFuzz_expireClaim_revertsBeforeDeadline() (gas: 313855)
[PASS] testFuzz_expireClaim_revertsWrongIndices() (gas: 316596)
```

```
[PASS] testFuzz_expireClaim_slashesUnsubmitted(uint8,uint8) (runs: 256, μ: 996307, ~: 984001)
[PASS] testFuzz_multipleClaims_independentSlots(uint8) (runs: 256, μ: 1712996, ~: 1367339)
[PASS] testFuzz_slotRecycling_afterExpiry(uint8) (runs: 256, μ: 526899, ~: 481892)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 1.19s (1.64s CPU time)

Ran 23 tests for test/libraries/ValidationLibFuzz.t.sol:ValidationLibFuzz
[PASS] testFuzz_batchCommitValidations_valid(uint8) (runs: 256, μ: 1474920, ~: 1232119)
[PASS] testFuzz_cancelExpiredValidationClaim_afterDeadline() (gas: 312260)
[PASS] testFuzz_cancelExpiredValidationClaim_revertsBeforeDeadline() (gas: 260070)
[PASS] testFuzz_claimToValidate_revertsEmpty() (gas: 28607)
[PASS] testFuzz_claimToValidate_revertsOwnContribution() (gas: 40014)
[PASS] testFuzz_claimToValidate_revertsTooManyValidators() (gas: 796987)
[PASS] testFuzz_claimToValidate_validClaim() (gas: 253036)
[PASS] testFuzz_claimToValidate_validWithOneSubmitted() (gas: 252596)
[PASS] testFuzz_commitValidation_revertsAfterClaimDeadline() (gas: 325337)
[PASS] testFuzz_commitValidation_revertsDoubleCommit() (gas: 468247)
[PASS] testFuzz_commitValidation_revertsNotClaimed() (gas: 94404)
[PASS] testFuzz_commitValidation_revertsZeroHash() (gas: 320946)
[PASS] testFuzz_commitValidation_revertsZeroStake() (gas: 265613)
[PASS] testFuzz_commitValidation_validCommit(bytes32,uint256,uint256) (runs: 256, μ: 445870, ~: 446010)
[PASS] testFuzz_computeConsensus_accepted(uint256,uint256,uint256) (runs: 256, μ: 1823196, ~: 1823242)
[PASS] testFuzz_computeConsensus_rejected(uint256,uint256,uint256) (runs: 256, μ: 1858201, ~: 1859037)
[PASS] testFuzz_computeConsensus_revertsAlreadyComputed() (gas: 1792271)
[PASS] testFuzz_computeConsensus_revertsNotEnoughReveals() (gas: 1006929)
[PASS] testFuzz_revealValidation_revertsAfterWindow() (gas: 460265)
[PASS] testFuzz_revealValidation_revertsDoubleReveal() (gas: 571126)
[PASS] testFuzz_revealValidation_revertsScoreOutOfRange(uint256) (runs: 256, μ: 450834, ~: 450738)
[PASS] testFuzz_revealValidation_revertsWrongHash(uint256,bytes32) (runs: 256, μ: 459360, ~: 459481)
[PASS] testFuzz_revealValidation_validReveal(uint256) (runs: 256, μ: 566904, ~: 567675)
Suite result: ok. 23 passed; 0 failed; 0 skipped; finished in 868.78ms (1.59s CPU time)

Ran 4 tests for test/lifecycle/ValidatorAlignment.t.sol:ValidatorAlignment
[PASS] test_dishonestValidatorEarnsNothingOverTime() (gas: 16212828)
[PASS] test_highStakeHonestValidatorResistsSybilAttack() (gas: 3398307)
[PASS] test_outlierValidatorDetectedAndSlashed() (gas: 4590768)
[PASS] test_splitOpinionResolvesAtThreshold() (gas: 2772358)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 22.23ms (17.10ms CPU time)

Ran 4 tests for test/audit/VerifyFixes.t.sol:VerifyFixesTest
[PASS] test_RISK003_settlementSucceedsOnRejection() (gas: 3120768)
[PASS] test_RISK005_escrowInsufficientForAllValidators() (gas: 3874749)
[PASS] test_RISK006_validatorLockedOutAfterResubmission() (gas: 4544565)
[PASS] test_RISK007_zeroStakeGetsWeight() (gas: 2101058)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 11.88ms (7.73ms CPU time)

Ran 11 tests for test/libraries/ReputationLibFuzz.t.sol:ReputationLibFuzz
[PASS] testFuzz_getScore_returnsDefaultForNewUser(address,bytes32) (runs: 256, μ: 29878, ~: 29878)
[PASS] testFuzz_reputation_crossDayAccumulation() (gas: 1934079)
[PASS] testFuzz_reputation_dailyGainCapped(uint8) (runs: 256, μ: 5688778, ~: 3913692)
[PASS] testFuzz_reputation_dailyGainDateTracking() (gas: 321118)
[PASS] testFuzz_reputation_decayAfterTime(uint8) (runs: 256, μ: 339231, ~: 339110)
[PASS] testFuzz_reputation_differentRolesIndependent(address) (runs: 256, μ: 50879, ~: 50879)
[PASS] testFuzz_reputation_lastUpdatedTimestamp(bytes32) (runs: 256, μ: 321013, ~: 321013)
[PASS] testFuzz_reputation_multipleProjectCreations(uint8) (runs: 256, μ: 1523997, ~: 1383776)
[PASS] testFuzz_reputation_scoreBoundedByMinMax(uint8) (runs: 256, μ: 8236674, ~: 5341339)
[PASS] testFuzz_reputation_updatedOnProjectCreation(bytes32) (runs: 256, μ: 327085, ~: 327085)
[PASS] testFuzz_reputation_zeroDecayNoChange(uint16) (runs: 256, μ: 333975, ~: 334151)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 1.57s (2.28s CPU time)

Ran 9 tests for test/findings/2026-02-24/SECURITY_FINDINGS_VERIFICATION.t.sol:SecurityFindings_2026_02_24
[PASS] test_SEC_C_01_settleValidatorDuringChallengePeriod() (gas: 2893749)
[PASS] test_SEC_H_01_escrowInsolvencyViaDisputeRejectionRecycle() (gas: 4960616)
[PASS] test_SEC_H_02_claimExpiryBlockedByRejectedRecycle() (gas: 2790682)
[PASS] test_SEC_H_03_immediateEscrowDrainAfterCancellation() (gas: 2750733)
[PASS] test_SEC_L_01_duplicateProjectCancelledEvent() (gas: 742613)
[PASS] test_SEC_L_02_reputationOldScoreInaccurate() (gas: 860229)
[PASS] test_SEC_M_01_removeProjectStrandsParticipantStakes() (gas: 935585)
[PASS] test_SEC_M_02_zeroChallengePeriodDisablesDisputes() (gas: 42060)
[PASS] test_SEC_M_03_settlementProceedsOnCancelledProject() (gas: 2551770)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 13.86ms (10.12ms CPU time)

Ran 7 tests for test/findings/2026-02-23/SECURITY_ISSUES_VERIFICATION.t.sol:SecurityIssuesVerification
```

```
[PASS] test_HIGH_01_ValidationClaimExpiryLocksValidatorSlots() (gas: 1575990)
[PASS] test_HIGH_02_LateCommitAllowedAfterValidationClaimExpiry() (gas: 1308658)
[PASS] test_HIGH_03_UpheldDisputesDeadlockProjectCompletion() (gas: 3333624)
[PASS] test_HIGH_04_FlashLoanConsensusManipulation() (gas: 16491)
[PASS] test_MEDIUM_01_CancelledProjectsStrandEscrow() (gas: 658672)
[PASS] test_MEDIUM_02_FeeStructureEconomicCentralization() (gas: 587762)
[PASS] test_MEDIUM_03_ERC20IntegrationAssumptions() (gas: 892658)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 8.93ms (5.45ms CPU time)

Ran 3 tests for test/findings/2026-02-
18/one/SEC_C_01_DisputeIndexPoisoning.t.sol:SEC_C_01_DisputeIndexPoisoning
[PASS] test_disputeFromOldNonceDoesNotAppearInGetDispute() (gas: 5181794)
[PASS] test_nonceKeyedDisputeDoesNotBlockNewContributor() (gas: 5189451)
[PASS] test_upheldDisputeDoesNotPoisonRecycledIndex() (gas: 5303547)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 13.75ms (10.03ms CPU time)

Ran 4 tests for test/findings/2026-02-
18/one/SEC_C_02_ReentrancyGuardStorage.t.sol:SEC_C_02_ReentrancyGuardStorage
[PASS] test_implementationHasUninitializedStatus() (gas: 3906923)
[PASS] test_initializerProperlyInitializesReentrancyGuard() (gas: 6532)
[PASS] test_proxyReentrancyStatusProperlyInitialized() (gas: 6680)
[PASS] test_proxyStatusRemainsNotEnteredAfterGuardedCall() (gas: 578133)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.37ms (785.42µs CPU time)

Ran 7 tests for test/economics/ECON_ProtocolEconomics.t.sol:ECON_ProtocolEconomics
[PASS] testECON_acceptedContributionSplitsRewardsAcrossRolesAndAdapter() (gas: 3366347)
[PASS] testECON_expiredCommitmentSlashesValidatorStake() (gas: 1464072)
[PASS] testECON_fundingWaterfallRoutesProtocolAndOriginationFees() (gas: 614358)
[PASS] testECON_generateRevenueModelDataFile() (gas: 7039485)
[PASS] testECON_originatorCanRecoverUnusedEscrowAfterCompletion() (gas: 3366088)
[PASS] testECON_pendingRewardsCanBeClaimedByContributorValidatorAndAdapter() (gas: 3365212)
[PASS] testECON_rejectedContributionSlashesContributorAndRecyclesSlot() (gas: 2758395)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 30.16ms (26.83ms CPU time)

Ran 3 tests for test/lifecycle/EconomicInvariants.t.sol:EconomicInvariants
[PASS] test_engineSolventAtEveryLifecycleStep() (gas: 3531231)
[PASS] test_escrowDrainEqualsRewardRate() (gas: 3318536)
[PASS] test_fundsConservedAcrossMixedOutcomes() (gas: 10957385)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 17.53ms (15.56ms CPU time)

Ran 16 tests for test/libraries/DisputeLibFuzz.t.sol:DisputeLibFuzz
[PASS] testFuzz_disputeOnRejectedContribution(uint256) (runs: 256, μ: 2752869, ~: 2754527)
[PASS] testFuzz_multipleDisputesAcrossProjects(uint8) (runs: 256, μ: 6839145, ~: 5278377)
[PASS] testFuzz_openDispute_bondCalculation(uint16) (runs: 256, μ: 2762522, ~: 2762374)
[PASS] testFuzz_openDispute_revertsAfterChallengePeriod(uint256) (runs: 256, μ: 57419, ~: 57436)
[PASS] testFuzz_openDispute_revertsContributorDisputing() (gas: 44683)
[PASS] testFuzz_openDispute_revertsDoubleDispute() (gas: 348056)
[PASS] testFuzz_openDispute_revertsNoConsensus(uint256) (runs: 256, μ: 38843, ~: 38872)
[PASS] testFuzz_openDispute_revertsZeroEvidenceHash() (gas: 30814)
[PASS] testFuzz_openDispute_validDispute(bytes32) (runs: 256, μ: 223750, ~: 223750)
[PASS] testFuzz_reportOriginator_blocksClaiming() (gas: 340317)
[PASS] testFuzz_reportOriginator_bondCalculation(uint16) (runs: 256, μ: 608887, ~: 608890)
[PASS] testFuzz_reportOriginator_revertsDoubleReport() (gas: 333248)
[PASS] testFuzz_reportOriginator_revertsNonExistentProject(bytes32) (runs: 256, μ: 30795, ~: 30795)
[PASS] testFuzz_reportOriginator_revertsOriginatorReporting() (gas: 32125)
[PASS] testFuzz_reportOriginator_revertsZeroEvidenceHash() (gas: 27762)
[PASS] testFuzz_reportOriginator_validReport(bytes32) (runs: 256, μ: 206800, ~: 206800)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 1.85s (2.89s CPU time)

Ran 19 tests for test/fuzz/FuzzExtended.t.sol:FuzzExtended
[PASS] testFuzz_adminFeesAffectAccounting(uint256,uint256) (runs: 256, μ: 882016, ~: 882229)
[PASS] testFuzz_batchContribute(uint256,uint256) (runs: 256, μ: 1535347, ~: 1648631)
[PASS] testFuzz_batchValidation(uint256,uint256) (runs: 256, μ: 3876788, ~: 3918322)
[PASS] testFuzz_cancelExpiredValidationClaim(uint256,uint256) (runs: 256, μ: 3084991, ~: 3122596)
[PASS] testFuzz_completeProjectAndRefundEscrow(uint256,uint256) (runs: 256, μ: 3317993, ~: 3275531)
[PASS] testFuzz_consensusManipulationVaryingStakeDistributions(uint256,uint256,uint256,uint256) (runs:
256, μ: 4935663, ~: 5213408)
[PASS] testFuzz_disputeEscalated(uint256,uint256) (runs: 256, μ: 2945112, ~: 2960293)
[PASS] testFuzz_disputeRejected_acceptedContribution(uint256,uint256) (runs: 256, μ: 3421030, ~: 3384843)
[PASS] testFuzz_disputeUpheld_acceptedContribution(uint256,uint256) (runs: 256, μ: 2983535, ~: 2998158)
[PASS] testFuzz_disputeUpheld_rejectedContribution(uint256,uint256) (runs: 256, μ: 3076482, ~: 3114229)
[PASS] testFuzz_erc20IntegrationAssumptions(uint256,uint256,uint256) (runs: 256, μ: 1024624, ~: 1025891)
```

```
[PASS] testFuzz_feeCalculationEdgeCases(uint256,uint256,uint256) (runs: 256, μ: 634234, ~: 636221)
[PASS] testFuzz_forceSettleUnresponsiveValidator(uint256,uint256) (runs: 256, μ: 3266276, ~: 3309011)
[PASS] testFuzz_lockedStakeUnavailableDuringClaim(uint256) (runs: 256, μ: 3226418, ~: 3226470)
[PASS] testFuzz_originatorReport_escalated(uint256) (runs: 256, μ: 811367, ~: 816511)
[PASS] testFuzz_originatorReport_rejected(uint256) (runs: 256, μ: 798968, ~: 802065)
[PASS] testFuzz_originatorReport_upheld(uint256) (runs: 256, μ: 813528, ~: 818176)
[PASS] testFuzz_reputationBounds(uint256,uint256,uint256) (runs: 256, μ: 6463388, ~: 7102860)
[PASS] testFuzz_varyingConsensusThreshold(uint256,uint256,uint256,uint256) (runs: 256, μ: 2724334, ~:
2761753)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 4.91s (9.19s CPU time)

Ran 10 tests for test/fuzz/Lifecycle.t.sol:LifecycleFuzzTest
[PASS] testFuzz_claimExpiration(uint256,uint256,uint256) (runs: 256, μ: 1444481, ~: 1424750)
[PASS] testFuzz_ghostValidatorSlash(uint256,uint256) (runs: 256, μ: 1454985, ~: 1455041)
[PASS] testFuzz_happyPathLifecycle(uint256,uint256,uint256,uint256,uint256,uint256) (runs: 256, μ:
3373275, ~: 3373341)
[PASS] testFuzz_mixedOutcomeSameProject(uint256,uint256,uint256,uint256) (runs: 256, μ: 4900692, ~:
4930355)
[PASS] testFuzz_multiContributionLifecycle(uint256,uint256,uint256,uint256,uint256,uint256) (runs: 256,
μ: 6291396, ~: 7041359)
[PASS] testFuzz_outlierValidatorSlashing(uint256,uint256,uint256,uint256) (runs: 256, μ: 4707965, ~:
4708200)
[PASS] testFuzz_rejectionLifecycle(uint256,uint256,uint256,uint256,uint256) (runs: 256, μ: 2789873, ~:
2790851)
[PASS] testFuzz_rejectionThenResubmission(uint256,uint256) (runs: 256, μ: 5288824, ~: 5327884)
[PASS] testFuzz_rewardAccountingInvariant(uint256,uint256,uint256,uint256,uint256) (runs: 256, μ:
3354967, ~: 3355023)
[PASS] testFuzz_stakeWeightedRewards(uint256,uint256,uint256,uint256,uint256) (runs: 256, μ: 3290647, ~:
3290711)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 5.49s (9.17s CPU time)

Ran 9 tests for test/invariant/SapienVaultInvariant.t.sol:SapienVaultInvariantTest
[PASS] invariant_availableBalanceConsistency() (runs: 256, calls: 3840, reverts: 0)

╭--------------------+------------------------+-------+---------+----------╮
| Contract           | Selector               | Calls | Reverts | Discards |
+=============================================================================+
| SapienVaultHandler | commitStake            | 373   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | deposit                | 400   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | lockContributor        | 400   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | lockValidatorCapacity  | 371   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | releaseCommit          | 372   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | slashContributor       | 348   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | slashValidator         | 399   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | unlockContributor      | 422   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | unlockValidatorCapacity | 363   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | withdraw               | 392   | 0       | 0        |
╰--------------------+------------------------+-------+---------+----------╯

[PASS] invariant_callSummary() (runs: 256, calls: 3840, reverts: 0)

╭--------------------+------------------------+-------+---------+----------╮
| Contract           | Selector               | Calls | Reverts | Discards |
+=============================================================================+
| SapienVaultHandler | commitStake            | 367   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | deposit                | 369   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | lockContributor        | 402   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | lockValidatorCapacity  | 385   | 0       | 0        |
|--------------------+------------------------+-------+---------+----------|
| SapienVaultHandler | releaseCommit          | 371   | 0       | 0        |
```

SapienVaultHandler	slashContributor	396	0	0
SapienVaultHandler	slashValidator	415	0	0
SapienVaultHandler	unlockContributor	356	0	0
SapienVaultHandler	unlockValidatorCapacity	378	0	0
SapienVaultHandler	withdraw	401	0	0

[PASS] invariant_depositWithdrawTokenConservation() (runs: 256, calls: 3840, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SapienVaultHandler	commitStake	394	0	0
SapienVaultHandler	deposit	399	0	0
SapienVaultHandler	lockContributor	383	0	0
SapienVaultHandler	lockValidatorCapacity	372	0	0
SapienVaultHandler	releaseCommit	344	0	0
SapienVaultHandler	slashContributor	372	0	0
SapienVaultHandler	slashValidator	388	0	0
SapienVaultHandler	unlockContributor	397	0	0
SapienVaultHandler	unlockValidatorCapacity	390	0	0
SapienVaultHandler	withdraw	401	0	0

[PASS] invariant_lockSolvency() (runs: 256, calls: 3840, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SapienVaultHandler	commitStake	395	0	0
SapienVaultHandler	deposit	415	0	0
SapienVaultHandler	lockContributor	382	0	0
SapienVaultHandler	lockValidatorCapacity	377	0	0
SapienVaultHandler	releaseCommit	362	0	0
SapienVaultHandler	slashContributor	420	0	0
SapienVaultHandler	slashValidator	359	0	0
SapienVaultHandler	unlockContributor	404	0	0
SapienVaultHandler	unlockValidatorCapacity	375	0	0
SapienVaultHandler	withdraw	351	0	0

[PASS] invariant_noZeroAddressBalance() (runs: 256, calls: 3840, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SapienVaultHandler	commitStake	402	0	0
SapienVaultHandler	deposit	371	0	0

SapienVaultHandler	lockContributor	394	0	0
SapienVaultHandler	lockValidatorCapacity	437	0	0
SapienVaultHandler	releaseCommit	412	0	0
SapienVaultHandler	slashContributor	334	0	0
SapienVaultHandler	slashValidator	391	0	0
SapienVaultHandler	unlockContributor	371	0	0
SapienVaultHandler	unlockValidatorCapacity	355	0	0
SapienVaultHandler	withdraw	373	0	0

[PASS] invariant_shareAssetConsistency() (runs: 256, calls: 3840, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SapienVaultHandler	commitStake	386	0	0
SapienVaultHandler	deposit	389	0	0
SapienVaultHandler	lockContributor	380	0	0
SapienVaultHandler	lockValidatorCapacity	353	0	0
SapienVaultHandler	releaseCommit	367	0	0
SapienVaultHandler	slashContributor	406	0	0
SapienVaultHandler	slashValidator	407	0	0
SapienVaultHandler	unlockContributor	353	0	0
SapienVaultHandler	unlockValidatorCapacity	403	0	0
SapienVaultHandler	withdraw	396	0	0

[PASS] invariant_slashOnlyBurnsShares() (runs: 256, calls: 3840, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SapienVaultHandler	commitStake	348	0	0
SapienVaultHandler	deposit	375	0	0
SapienVaultHandler	lockContributor	410	0	0
SapienVaultHandler	lockValidatorCapacity	375	0	0
SapienVaultHandler	releaseCommit	370	0	0
SapienVaultHandler	slashContributor	382	0	0
SapienVaultHandler	slashValidator	374	0	0
SapienVaultHandler	unlockContributor	391	0	0
SapienVaultHandler	unlockValidatorCapacity	397	0	0
SapienVaultHandler	withdraw	418	0	0

[PASS] invariant_vaultTokenSolvency() (runs: 256, calls: 3840, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SapienVaultHandler	commitStake	369	0	0
SapienVaultHandler	deposit	417	0	0
SapienVaultHandler	lockContributor	389	0	0
SapienVaultHandler	lockValidatorCapacity	375	0	0
SapienVaultHandler	releaseCommit	374	0	0
SapienVaultHandler	slashContributor	412	0	0
SapienVaultHandler	slashValidator	389	0	0
SapienVaultHandler	unlockContributor	362	0	0
SapienVaultHandler	unlockValidatorCapacity	377	0	0
SapienVaultHandler	withdraw	376	0	0

[PASS] invariant_withdrawalGuard() (runs: 256, calls: 3840, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SapienVaultHandler	commitStake	357	0	0
SapienVaultHandler	deposit	385	0	0
SapienVaultHandler	lockContributor	406	0	0
SapienVaultHandler	lockValidatorCapacity	375	0	0
SapienVaultHandler	releaseCommit	399	0	0
SapienVaultHandler	slashContributor	406	0	0
SapienVaultHandler	slashValidator	362	0	0
SapienVaultHandler	unlockContributor	388	0	0
SapienVaultHandler	unlockValidatorCapacity	390	0	0
SapienVaultHandler	withdraw	372	0	0

Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 4.54s (15.01s CPU time)

Ran 20 tests for test/libraries/OriginationLibFuzz.t.sol:OriginationLibFuzz
[PASS] testFuzz_createProject_revertsDuplicateId(bytes32) (runs: 256, μ: 321796, ~: 321796)
[PASS] testFuzz_createProject_revertsHighValidatorReward(bytes32,uint256) (runs: 256, μ: 34648, ~: 34582)
[PASS] testFuzz_createProject_revertsInvalidConsensusThreshold(bytes32,uint256) (runs: 256, μ: 33916, ~: 33798)
[PASS] testFuzz_createProject_revertsInvalidNumberOfValidations(bytes32,uint256) (runs: 256, μ: 33851, ~: 33838)
[PASS] testFuzz_createProject_revertsWrongOriginator(bytes32,address) (runs: 256, μ: 31306, ~: 31306)
[PASS] testFuzz_createProject_revertsZeroConsensusThreshold(bytes32) (runs: 256, μ: 30888, ~: 30888)
[PASS] testFuzz_createProject_revertsZeroRewardToken(bytes32) (runs: 256, μ: 30652, ~: 30652)
[PASS] testFuzz_createProject_revertsZeroValidations(bytes32) (runs: 256, μ: 30923, ~: 30923)
[PASS] testFuzz_createProject_validConfigurations(bytes32,uint16,uint16,uint8) (runs: 256, μ: 338112, ~: 339413)
[PASS] testFuzz_fundProject_multipleFundings(bytes32,uint8) (runs: 256, μ: 606430, ~: 598722)
[PASS] testFuzz_fundProject_originationAdapterFee(bytes32,address) (runs: 256, μ: 590098, ~: 590098)
[PASS] testFuzz_fundProject_protocolFeeDeduction(bytes32,uint16) (runs: 256, μ: 543668, ~: 546073)
[PASS] testFuzz_fundProject_revertsNonOriginator(bytes32,address) (runs: 256, μ: 384158, ~: 384158)
[PASS] testFuzz_fundProject_revertsZeroAmount(bytes32,uint256) (runs: 256, μ: 328730, ~: 328791)
[PASS] testFuzz_fundProject_revertsZeroQuantity(bytes32,uint256) (runs: 256, μ: 329376, ~: 329206)
[PASS] testFuzz_fundProject_validFunding(bytes32,uint256,uint256) (runs: 256, μ: 545294, ~: 545383)

```
[PASS] testFuzz_fundProject_withOriginatorStakeRequirement(bytes32,uint256,uint256) (runs: 256, μ:
748471, ~: 748523)
[PASS] testFuzz_removeProject_revertsAlreadyCancelled(bytes32) (runs: 256, μ: 590470, ~: 590470)
[PASS] testFuzz_removeProject_revertsNonExistent(bytes32) (runs: 256, μ: 32611, ~: 32611)
[PASS] testFuzz_removeProject_slashesOriginator(bytes32) (runs: 256, μ: 810074, ~: 810074)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 5.15s (1.52s CPU time)

Ran 13 tests for test/invariant/SapienCoreInvariant.t.sol:SapienCoreInvariantTest
[PASS] invariant_callSummary() (runs: 256, calls: 3840, reverts: 9)

╭------------------+--------------------------+-------+---------+----------╮
| Contract         | Selector                 | Calls | Reverts | Discards |
+=============================================================================+
| SapienCoreHandler | claimAndContribute       | 530   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | claimReward              | 549   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | commitValidation         | 549   | 9       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | computeConsensus         | 522   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | createAndFundProject     | 572   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | releaseContributorReward | 552   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | settleValidator          | 566   | 0       | 0        |
╰------------------+--------------------------+-------+---------+----------╯

[PASS] invariant_contributionStatusConsistency() (runs: 256, calls: 3840, reverts: 12)

╭------------------+--------------------------+-------+---------+----------╮
| Contract         | Selector                 | Calls | Reverts | Discards |
+=============================================================================+
| SapienCoreHandler | claimAndContribute       | 523   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | claimReward              | 565   | 1       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | commitValidation         | 540   | 10      | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | computeConsensus         | 557   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | createAndFundProject     | 555   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | releaseContributorReward | 566   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | settleValidator          | 534   | 1       | 0        |
╰------------------+--------------------------+-------+---------+----------╯

[PASS] invariant_engineBalanceSanity() (runs: 256, calls: 3840, reverts: 2)

╭------------------+--------------------------+-------+---------+----------╮
| Contract         | Selector                 | Calls | Reverts | Discards |
+=============================================================================+
| SapienCoreHandler | claimAndContribute       | 577   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | claimReward              | 533   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | commitValidation         | 524   | 2       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | computeConsensus         | 544   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | createAndFundProject     | 559   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | releaseContributorReward | 559   | 0       | 0        |
|------------------+--------------------------+-------+---------+----------|
| SapienCoreHandler | settleValidator          | 544   | 0       | 0        |
╰------------------+--------------------------+-------+---------+----------╯

[PASS] invariant_engineTokenSolvency() (runs: 256, calls: 3840, reverts: 9)

╭------------------+--------------------------+-------+---------+----------╮
| Contract         | Selector                 | Calls | Reverts | Discards |
```

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	557	0	0
SapienCoreHandler	claimReward	540	0	0
SapienCoreHandler	commitValidation	553	8	0
SapienCoreHandler	computeConsensus	517	0	0
SapienCoreHandler	createAndFundProject	563	0	0
SapienCoreHandler	releaseContributorReward	554	0	0
SapienCoreHandler	settleValidator	556	1	0

[PASS] invariant_feeBounds() (runs: 256, calls: 3840, reverts: 10)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	537	0	0
SapienCoreHandler	claimReward	531	0	0
SapienCoreHandler	commitValidation	548	10	0
SapienCoreHandler	computeConsensus	576	0	0
SapienCoreHandler	createAndFundProject	552	0	0
SapienCoreHandler	releaseContributorReward	542	0	0
SapienCoreHandler	settleValidator	554	0	0

[PASS] invariant_projectEscrowReasonable() (runs: 256, calls: 3840, reverts: 7)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	533	0	0
SapienCoreHandler	claimReward	554	0	0
SapienCoreHandler	commitValidation	580	7	0
SapienCoreHandler	computeConsensus	527	0	0
SapienCoreHandler	createAndFundProject	526	0	0
SapienCoreHandler	releaseContributorReward	548	0	0
SapienCoreHandler	settleValidator	572	0	0

[PASS] invariant_projectSlotAccounting() (runs: 256, calls: 3840, reverts: 11)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	513	0	0
SapienCoreHandler	claimReward	556	0	0
SapienCoreHandler	commitValidation	542	11	0
SapienCoreHandler	computeConsensus	521	0	0
SapienCoreHandler	createAndFundProject	576	0	0
SapienCoreHandler	releaseContributorReward	571	0	0

| SapienCoreHandler | settleValidator | 561 | 0 | 0 |

[PASS] invariant_reputationBounds() (runs: 256, calls: 3840, reverts: 9)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	583	0	0
SapienCoreHandler	claimReward	550	0	0
SapienCoreHandler	commitValidation	542	9	0
SapienCoreHandler	computeConsensus	534	0	0
SapienCoreHandler	createAndFundProject	531	0	0
SapienCoreHandler	releaseContributorReward	569	0	0
SapienCoreHandler	settleValidator	531	0	0

[PASS] invariant_reputationDailyGainCap() (runs: 256, calls: 3840, reverts: 10)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	565	0	0
SapienCoreHandler	claimReward	534	0	0
SapienCoreHandler	commitValidation	522	9	0
SapienCoreHandler	computeConsensus	610	0	0
SapienCoreHandler	createAndFundProject	537	0	0
SapienCoreHandler	releaseContributorReward	534	0	0
SapienCoreHandler	settleValidator	538	1	0

[PASS] invariant_stateConsistency() (runs: 256, calls: 3840, reverts: 5)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	555	0	0
SapienCoreHandler	claimReward	574	0	0
SapienCoreHandler	commitValidation	495	5	0
SapienCoreHandler	computeConsensus	547	0	0
SapienCoreHandler	createAndFundProject	547	0	0
SapienCoreHandler	releaseContributorReward	561	0	0
SapienCoreHandler	settleValidator	561	0	0

[PASS] invariant_tokenConservation() (runs: 256, calls: 3840, reverts: 8)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	562	0	0
SapienCoreHandler	claimReward	588	0	0

SapienCoreHandler	commitValidation	546	8	0
SapienCoreHandler	computeConsensus	526	0	0
SapienCoreHandler	createAndFundProject	519	0	0
SapienCoreHandler	releaseContributorReward	534	0	0
SapienCoreHandler	settleValidator	565	0	0

[PASS] invariant_validatorRewardBpsBounds() (runs: 256, calls: 3840, reverts: 13)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	541	0	0
SapienCoreHandler	claimReward	567	0	0
SapienCoreHandler	commitValidation	528	13	0
SapienCoreHandler	computeConsensus	529	0	0
SapienCoreHandler	createAndFundProject	567	0	0
SapienCoreHandler	releaseContributorReward	584	0	0
SapienCoreHandler	settleValidator	524	0	0

[PASS] invariant_vaultLockSolvencyThroughProtocol() (runs: 256, calls: 3840, reverts: 15)

Contract	Selector	Calls	Reverts	Discards
SapienCoreHandler	claimAndContribute	507	0	0
SapienCoreHandler	claimReward	597	0	0
SapienCoreHandler	commitValidation	536	15	0
SapienCoreHandler	computeConsensus	550	0	0
SapienCoreHandler	createAndFundProject	549	0	0
SapienCoreHandler	releaseContributorReward	535	0	0
SapienCoreHandler	settleValidator	566	0	0

Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 5.79s (26.93s CPU time)

Ran 15 tests for test/libraries/ConsensusLibFuzz.t.sol:ConsensusLibFuzz
[PASS] testFuzz_calculate_accurateWeightTracking(uint8,uint8) (runs: 256, μ: 71812, ~: 64863)
[PASS] testFuzz_calculate_allOutliersExceptOne(uint8,uint256) (runs: 256, μ: 59344, ~: 53229)
[PASS] testFuzz_calculate_consistentWeightOrdering(uint256,uint256,uint256,uint256) (runs: 256, μ: 29056, ~: 29187)
[PASS] testFuzz_calculate_deterministicResults(address[3],uint256[3],uint256[3],uint256[3]) (runs: 256, μ: 73366, ~: 73561)
[PASS] testFuzz_calculate_extremeReputationValues(uint256) (runs: 256, μ: 25711, ~: 25858)
[PASS] testFuzz_calculate_extremeScoreDifference(uint256,uint256,uint256,uint256) (runs: 256, μ: 30655, ~: 30696)
[PASS] testFuzz_calculate_manyValidatorsNoOverflow(uint8,uint256) (runs: 256, μ: 110589, ~: 85306)
[PASS] testFuzz_calculate_maxStakeNoOverflow(uint256) (runs: 256, μ: 24382, ~: 24537)
[PASS] testFuzz_calculate_outlierDetection(uint256,uint256,uint256) (runs: 256, μ: 38057, ~: 39458)
[PASS] testFuzz_calculate_singleValidator(address,uint256,uint256,uint256) (runs: 256, μ: 26411, ~: 26232)
[PASS] testFuzz_calculate_tieredSlashing(uint256,uint256) (runs: 256, μ: 67640, ~: 67652)
[PASS] testFuzz_calculate_twoValidatorsSameScore(address,address,uint256,uint256,uint256,uint256,uint256) (runs: 256, μ: 32528, ~: 32681)

```
[PASS] testFuzz_calculate_zeroReputationUsesFloor(uint256,uint256) (runs: 256, μ: 22054, ~: 21996)
[PASS] testFuzz_calculate_zeroStakeHandled(uint256,uint256) (runs: 256, μ: 21184, ~: 21108)
[PASS] test_calculate_revertsOnEmptyInputs() (gas: 11096)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 6.39s (6.69s CPU time)

Ran 87 test suites in 6.47s (41.09s CPU time): 682 tests passed, 0 failed, 0 skipped (682 total tests)
```

Code Coverage

It was not possible to generate coverage information.

Changelog

- 2026-03-06 - Initial report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, PolyMarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.